SPEC- 1 -Unified Payment Collection and Receipt Management App

Background

Businesses increasingly receive payments across multiple digital payment apps, but the operation work that follows is still fragmented. Payment records arrive in different formats, with incons customer identifiers, varying settlement signals, and uneven API/webhook support. Teams then reconcile transactions manually, search across provider dashboards, and issue receipts or payme confirmations through separate invoicing tools.

The attached SRS and SDD define the product as an orchestration and visibility platform rath funds-movement system. Its core responsibility is to ingest payment events from multiple provi normalize them into a canonical ledger, deduplicate repeats, let operators review and enrich transactions, and trigger document issuance through compliant third-party invoicing services.

For the MVP, the design target is a multi-tenant SaaS platform for SMBs and finance/operatio with Israeli-compatible payment and invoicing ecosystems in scope. The documents strongly favor canonical transaction domain - adapter isolation for provider-specific behavior - duplicate-safe synchronization and receipt issuance - auditable user and system actions - a responsive back dashboard for non-technical staff

The architecture should therefore optimize for correctness, traceability, and integration flexibility be large-scale service decomposition. A modular monolith with background workers is the most appropriate starting point because it keeps implementation complexity manageable while preserving clean migration path toward separate services if volume, team size, or provider complexity grows.

Requirements

Must have

- Multi-tenant business accounts with strict tenant isolation.
- Mobile-first user experience for business operators, with a responsive admin web console for setup, support, and audit workflows.
- Secure authentication, session/token management, and role-based access control for owner/admin/operator roles.
- Integration with at least two payment sources in the MVP, using provider adapters behind a common interface.
- Support webhook-based ingestion where available and scheduled polling where required by the provider.
- Canonical transaction ledger that normalizes provider-specific payment records into a common internal model.
- Duplicate-safe ingestion using provider transaction IDs, idempotency keys, sync cursors, and fingerprint heuristics.
- Transaction search, sort, and filtering across core dimensions such as provider, date, amount, status, customer, tag, and receipt state.

- Transaction detail view with raw-source traceability, normalization results, customer linkage, tags, and workflow history.
- Receipt / payment-confirmation issuance via at least one compliant invoicing provider in the MVP.
- Duplicate-safe document issuance with idempotent issue jobs and conflict checks.
- Business profile and branding settings that propagate to supported receipt/document workflows.
- Audit logs for sync, data changes, issuance actions, retries, failures, and user activity.
- Background job processing for sync, retries, notification delivery, and export generation.
- Secure storage of provider credentials and API secrets outside source code.
- Observability baseline: structured logs, core operational metrics, retry visibility, and failure alerts.

Should have

- Push notifications for sync failures, issuance failures, and action-required events in the mobile app.
- Offline-tolerant mobile behavior for transaction browsing and queued user actions, with safe re-sync on reconnect.
- Customer directory with basic matching/linking of transactions to customers.
- Bulk actions for tagging, exports, and retrying failed workflows.
- Integration health status screen showing credential validity, last sync time, and recent provider errors.
- Export of transactions and audit data for finance/operations use.
- Delivery of issued documents to customers through email or messaging integrations where supported.
- Immutable raw payload archive in object storage for troubleshooting and audit.
- Dead-letter handling for permanently failing sync or issuance jobs.

Could have

- Manual CSV/import adapter for providers without usable APIs.
- Smart customer matching suggestions based on payer name, phone, email, or provider metadata.
- Tagging rules or automation for transaction categorization.
- In-app operational notes and exception handling workflows.
- Multi-language UX support, including Hebrew-first localization.
- Analytics views for provider performance, settlement trends, and issuance throughput.

Won't have for MVP

- Native legal invoice generation without an external compliant invoicing provider.
- Funds movement, payout orchestration, wallet balances, or payment processing.
- Full microservices decomposition from day one.
- Deep accounting/ERP bidirectional synchronization beyond minimal exports or basic adapter hooks.
- Complex AI-driven reconciliation or fraud detection in the first release.

Method

The MVP architecture will be implemented as an API-first, mobile-first system on GCP using a backend design. The system should keep provider-specific logic isolated behind adapters, while internal application behavior works against canonical domain models.

Architecture overview

Primary components: - Flutter mobile app for day-to-day business operations. - Small React/Ne admin web console for setup, support, integration management, and audit visibility. - FastAPI b deployed on Cloud Run. - Firestore as the primary operational database. - Cloud Storage f provider payloads, exported files, and generated documents. - Cloud Tasks for retryable job ex and delayed workflows. - Pub/Sub for domain event fan-out and asynchronous side effects. - Scheduler for recurring sync triggers. - Simple application-managed authentication for MVP usin email/password, with a planned migration path to Google OAuth and later phone-based OTP. notification capability reserved for a later milestone; initial MVP can rely on in-app status ar alerts.

Architectural style

Use a modular monolith at the codebase level, split into bounded modules: - auth - tenants transactions - customers - documents - notifications - audit - reporting

Each module exposes internal service interfaces and owns its validation, persistence mapping, domain rules. Provider adapters are plug-ins under the providers module.

This keeps the initial deployment simple while preserving future extraction paths. The first pro deployment can be a small set of Cloud R`api-service`: - synchronous HTTP API for mobile app, admin web, and provider `worker-service`: HTTP handlers invoked by Cloud Tasks for retries, polling, document issuance, and heavy `event-service` (optional in MVP): Pub/Sub push subscriber endpoints for event fan-out consumers

High-level flow

1. Mobile app or admin console authenticates and calls FastAPI APIs.
2. Provider sends a webhook, or Cloud Scheduler triggers a sync task.
3. Backend retrieves provider data through an adapter.
4. Raw payload is archived in Cloud Storage.
5. Payload is normalized into canonical transaction records.
6. Deduplication and idempotency rules decide whether to insert, merge, or ignore.
7. Firestore transaction document and audit events are written.
8. Business rules may enqueue document issuance or notifications.
9. Cloud Tasks executes retryable workflows; Pub/Sub distributes non-critical side effects.
10. Mobile users see updated transaction state and receive push notifications when required.

PlantUML component diagram

```
@startuml
actor BusinessUser
```

```
actor AdminUser
actor PaymentProvider
actor InvoicingProvider

node "Flutter Mobile App" as Mobile
node "Admin Web" as AdminWeb
node "Cloud Run: API Service" as API
node "Cloud Run: Worker Service" as Worker
queue "Cloud Tasks" as Tasks
queue "Pub/Sub" as PubSub
database "Firestore" as Firestore
folder "Cloud Storage" as GCS
node "Cloud Scheduler" as Scheduler
node "FCM" as FCM

BusinessUser --> Mobile
AdminUser --> AdminWeb
Mobile --> API
AdminWeb --> API
PaymentProvider --> API : Webhooks
Scheduler --> API : Sync trigger
API --> Firestore
API --> GCS
API --> Tasks
API --> PubSub
Tasks --> Worker
Worker --> Firestore
Worker --> GCS
Worker --> InvoicingProvider
PubSub --> Worker
API --> FCM
@enduml
```

Core domain model

Because Firestore is document-oriented, the design should use aggregate-centric documents and denormalized read fields where necessary.

Main collections

`tenants/{tenantId}`

Represents a business account.

Suggested fields: `tenantId` `name` `legalName` `countryCode` `timezone` `currency` - `status` `active|suspended|pending`) - `branding` `{logoUrl, primaryColor, senderName}` - `defaultDocumentProviderId` - `createdAt` - `updatedAt`

`users/{userId}`

Global user identity mapped to one or more tenant memberships.

Suggested fields: `userId` `email` `phone` `displayName` `status` `lastLoginAt` - `createdAt`

`tenant_memberships/{tenantId_userId}`

Authorization mapping.

Suggested fields: `tenantId` - `userId` - `role` `owner|admin|operator|viewer`) - `permissions` `string[]` - `createdAt` - `updatedAt`

`provider_connections/{connectionId}`

One configured integration for a tenant.

Suggested fields: `connectionId` - `tenantId` - `providerType` `bit|paybox|cardcom|greeninvoice|...`) - `category` `payment_source|document_provider`) - `displayName` - `status` `active|error|reauth_required|disabled`) `credentialsRef` (reference to Secret Manager secret path) `webhookConfig` `{enabled, externalId, lastValidatedAt}` - `syncConfig` `{mode, cursor, lastSyncAt, nextSyncAt}` `health` `{lastSuccessAt, lastErrorCode, lastErrorMessage}` - `createdAt` - `updatedAt`

`transactions/{transactionId}`

Canonical payment transaction.

Suggested fields: `transactionId` `tenantId` `providerType` `providerConnectionId` - `providerTransactionId` - `providerEventId` - `idempotencyKey` - `fingerprintHash` - `status` `pending|succeeded|failed|refunded|cancelled|unknown`) - `amount` `{valueMinor, currency}` `occurredAt` `capturedAt` `payer` `{name, phone, email, externalRef}` - `customerId` - `description` - `tags` `string[]` - `receiptState` (`not_required|pending|issued|failed`) - `documentId` `sourceSummary` `{channel, settlementRef, rawPayloadPath}` - `search` `{payerNameLower, phoneNormalized, amountBucket, dateKey}` - `createdAt` - `updatedAt`

Recommended key strategy: - Primary document ID may be a generated ULID. - Enforce un through an auxiliary dedup collection rather than relying on document ID alone.

`transaction_dedup/{tenantId_hash}`

Used to prevent duplicates.

Suggested fields: - `tenantId` - `hashType` `provider_txn|event_id|fingerprint`) - `hashValue` - `transactionId` - `createdAt`

Document ID example: - `${tenantId}_${hashType}_${sha256(hashValue)}`

`customers/{customerId}`

Canonical customer directory.

Suggested fields: `customerId` `tenantId` `name` `phone` `email` `externalRefs` `object` - `tags` `string[]` - `createdAt` - `updatedAt`

`documents/{documentId}`

Tracks issued receipts or payment confirmations.

Suggested fields: `documentId` - `tenantId` - `transactionId` - `providerType` - `providerDocumentId` `documentType` `receipt|payment_confirmation|invoice_receipt`) - `status` `queued|issued|delivered|failed`) - `issueRequestHash` - `filePath` - `delivery` `{channel, recipient, deliveredAt, status}` `error` `{code, message}` - `createdAt` - `updatedAt`

`audit_events/{eventId}`

Immutable operational and user audit stream.

Suggested fields: `eventId` `tenantId` `entityType` `entityId` `eventType` `actorType` (`user|system|provider`) - `actorId` - `correlationId` - `payload` - `createdAt`

`sync_runs/{syncRunId}`

Tracks each polling or webhook-driven synchronization run.

Suggested fields: `syncRunId` - `tenantId` - `providerConnectionId` - `triggerType` (`webhook|scheduled|manual|retry`) - `status` `running|succeeded|partial|failed`) - `startedAt` - `finishedAt` - `cursorBefore` - `cursorAfter` - `stats` `{fetched, inserted, merged, ignored, failed}` - `errorSummary`

Detailed Firestore schema and indexes

Firestore is the highest-risk persistence choice in this architecture because the application beha partly like a ledger, partly like an operations console, and partly like an integration hub. Th must therefore be optimized around: - predictable filtered reads - low write amplification - atomic operations - explicit denormalization - controlled index growth

Google's Firestore guidance is directly relevant here: query performance depends on indexes, composite indexes are limited per database, index fanout increases write latency and storage queries usin `or` `in` `array-contains-any` are limited by disjunctive-normal-form expansion, and transactions should remain narrow. (firebase.google.com)

Firestore document design principles

- Keep top-level collections for core aggregates instead of nesting business-critical data deeply.
- Model the write path first, then build read models that match exact UI/API filters.
- Do not rely on ad hoc querying for finance workflows; define approved query shapes up front.

- Minimize array fields that need querying because array indexing adds write fanout.
- Exempt fields from indexing if they are large or never queried, especially verbose payload summaries or notes, because Firestore best practices call out index fanout as a major driver of write latency and cost. (firebase.google.com)
- Keep immutable or large raw payloads in Cloud Storage, with only references in Firestore.
- Use server-side timestamps on all persistent operational records.
- Limit Firestore transactions to the smallest possible number of documents; Firestore supports atomic transactions and batched writes, but narrow usage reduces contention risk. (firebase.google.com)

Recommended top-level collections

```
/tenants/{tenantId}
/users/{userId}
/tenant_memberships/{tenantId_userId}
/auth_sessions/{sessionId}
/provider_connections/{connectionId}
/transactions/{transactionId}
/transaction_dedup/{dedupKey}
/customers/{customerId}
/documents/{documentId}
/audit_events/{eventId}
/sync_runs/{syncRunId}
/outbox_events/{eventId}
```

Use top-level collections rather than subcollections for the high-volu `transactions`,(`documents`, `audit_events`, `sync_runs`) because the admin and mobile experiences both need tenant-scoped operational queries across broad record sets.

Collection details and field-level index strategy

`tenants/{tenantId}`

Purpose: - business account root - branding and defaults - high-read, low-write

Fields:

```
{
 "tenantId": "t_01H...",
 "name": "Acme Payments",
 "legalName": "Acme Ltd",
 "countryCode": "IL",
 "timezone": "Asia/Jerusalem",
 "currency": "ILS",
 "status": "active",
 "branding": {
  "logoUrl": "gs://...",
  "primaryColor": "#123456",
```

```
  "senderName": "Acme"
 },
 "defaultDocumentProviderId": "pc_123",
 "createdAt": "timestamp",
 "updatedAt": "timestamp"
}
```

Indexing: - default single-field indexes are enough - no composite indexes needed initially - `branding.logoUrl` if not queried directly

`users/{userId}`

Purpose: - identity profile for app-managed auth

Fields:

```
{
 "userId": "u_01H...",
 "email": "owner@example.com",
 "emailLower": "owner@example.com",
 "displayName": "Jane Doe",
 "phone": "+972...",
 "passwordHash": "argon2_hash",
 "googleAccountId": null,
 "phoneVerified": false,
 "phoneVerifiedAt": null,
 "authMethods": ["password"],
 "status": "active",
 "lastLoginAt": "timestamp",
 "createdAt": "timestamp"
}
```

Indexing: - query `emailLower` - disable indexin `passwordHash` - disable indexing on rarely queried profile fields

Recommended lookup: - `where(emailLower == ?).limit(1)`

`tenant_memberships/{tenantId_userId}`

Purpose: - tenant-scoped role lookup

Document ID: - `${tenantId}_${userId}`

Fields:

```
{
 "tenantId": "t_...",
```

```
  "userId": "u_...",
  "role": "owner",
  "permissions": ["transactions.read", "documents.issue"],
  "status": "active",
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
}
```

Indexing: - composite i`(tenantId, userId)` only if querying by fields rather than direct doc - if doc ID is always known, no composite needed - `permissions` yings as an array; permissions should be evaluated in application memory after membership fetch

`auth_sessions/{sessionId}`

Purpose: - refresh token tracking and revocation

Fields:

```
{
  "sessionId": "s_01H...",
  "userId": "u_...",
  "tenantId": "t_...",
  "refreshTokenHash": "sha256...",
  "deviceInfo": {
    "platform": "ios",
    "appVersion": "1.0.0"
  },
  "ipAddressLastSeen": "1.2.3.4",
  "status": "active",
  "createdAt": "timestamp",
  "expiresAt": "timestamp",
  "revokedAt": null
}
```

Indexing: - composite i`(userId, status, createdAt desc)` for session management UI - optionally TTL/cleanup by `expiresAt`

`provider_connections/{connectionId}`

Purpose: - one provider integration instance for a tenant

Fields:

```
{
  "connectionId": "pc_01H...",
  "tenantId": "t_...",
  "providerType": "bit",
```

```
  "category": "payment_source",
  "displayName": "Bit Main Account",
  "status": "active",
  "credentialsRef": "projects/.../secrets/...",
  "webhookConfig": {
   "enabled": true,
   "externalId": "wh_123",
   "lastValidatedAt": "timestamp"
  },
  "syncConfig": {
   "mode": "webhook_plus_poll",
   "cursor": "opaque_cursor",
   "lastSyncAt": "timestamp",
   "nextSyncAt": "timestamp"
  },
  "health": {
   "lastSuccessAt": "timestamp",
   "lastErrorCode": null,
   "lastErrorMessage": null
  },
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
 }
```

Indexing: - composite in `(tenantId, category, status)` - composite in `(tenantId, providerType, status)` - composite inc `(status, syncConfig.nextSyncAt)` only if querying Firestore for due jobs; preferably avoid this by letting Cloud Scheduler/Tasks own scheduling - exempt `health.lastErrorMessage` and `syncConfig.cursor` from indexing

`transactions/{transactionId}`

Purpose: - canonical ledger-like transaction record - highest-volume operational collection

Fields:

```
 {
  "transactionId": "txn_01H...",
  "tenantId": "t_...",
  "providerType": "bit",
  "providerConnectionId": "pc_...",
  "providerTransactionId": "ext_123",
  "providerEventId": "evt_123",
  "idempotencyKey": "idemp_...",
  "fingerprintHash": "sha256...",
  "status": "succeeded",
  "amount": {
   "valueMinor": 1050,
   "currency": "ILS"
```

```
  },
  "occurredAt": "timestamp",
  "capturedAt": "timestamp",
  "payer": {
   "name": "John Doe",
   "nameLower": "john doe",
   "phone": "+972...",
   "phoneNormalized": "9725...",
   "email": "john@example.com",
   "emailLower": "john@example.com",
   "externalRef": null
  },
  "customerId": "cust_...",
  "description": "Order #1234",
  "tags": ["vip", "manual-review"],
  "receiptState": "issued",
  "documentId": "doc_...",
  "sourceSummary": {
   "channel": "webhook",
   "settlementRef": "set_123",
   "rawPayloadPath": "gs://bucket/raw/...json"
  },
  "search": {
   "dateKey": "2026-03-09",
   "monthKey": "2026-03",
   "amountBucket": 1000,
   "payerNamePrefix": "john",
   "hasPhone": true,
   "hasEmail": true
  },
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
 }
```

Indexing and query policy:

Supported MVP queries should be explicitly limited to these shapes: - tenant + recent trans ordered b`occurredAt desc` - tenant + provider + recent transactions - tenant + status + transactions - tenant + receiptState + recent transactions - tenant + date range + recent tra tenant + customerId + recent transactions - tenant + exact normalized payer phone - tenant normalized payer email

Recommended composite indexe`(tenantId asc, occurredAt desc)` `(tenantId asc, providerType asc, occurredAt desc)` `(tenantId asc, status asc, occurredAt desc)` - `(tenantId asc, receiptState asc, occurredAt desc)` `(tenantId asc, customerId asc, occurredAt desc)` `(tenantId asc, payer.phoneNormalized asc, occurredAt desc)` `(tenantId asc, payer.emailLower asc, occurredAt desc)` `(tenantId asc, amount.currency asc, occurredAt desc)` only if multi-currency list screens are needed

Optional composite indexes only if the UI really needs them: - `(tenantId asc, providerType asc, status asc, occurredAt desc)` `(tenantId asc, search.monthKey asc, occurredAt desc)`

Do not support in MVP: - arbitrary multi-field search combinatorics - free-text contains search payer/description - large OR-heavy dashboards us`in` `or`y combinations, because Firestore limits disjunction expansion to `30` and limits inequality/range complexity. (firebase.google.com)

Index exemptions: - exe`sourceSummary.rawPayloadPath` - exem`description` if only displayed, not queried - e`tags` from indexing unless tag filtering is truly needed in MVP exempt unused nested payer fields if not queried

Important search decision: - For text search, use exact-match normalized fields only in MVP. substring/full-text search becomes important later, add a dedicated search service rather than stretching Firestore beyond its strengths.

`transaction_dedup/{dedupKey}`

Purpose: - uniqueness reservation for webhook + polling overlap

Document ID: - `${tenantId}_${hashType}_${sha256(hashValue)}`

Fields:

```
{
 "tenantId": "t_...",
 "hashType": "provider_txn",
 "hashValue": "ext_123",
 "transactionId": "txn_...",
 "createdAt": "timestamp"
}
```

Indexing: - direct doc lookup by ID only - no composite indexes needed

Write pattern: - use Firestore trans`transaction_dedup` reservation p`transactions` create/update pointer write - keep this transaction narrow to reduce contention risk. (firebase.google.com)

`customers/{customerId}`

Purpose: - canonical tenant customer directory

Fields:

```
{
 "customerId": "cust_01H...",
 "tenantId": "t_...",
```

```
  "name": "John Doe",
  "nameLower": "john doe",
  "phone": "+972...",
  "phoneNormalized": "9725...",
  "email": "john@example.com",
  "emailLower": "john@example.com",
  "externalRefs": {
   "bitPayerId": "abc"
  },
  "tags": ["vip"],
  "createdAt": "timestamp",
  "updatedAt": "timestamp"
 }
```

Recommended composite indexes: - `(tenantId asc, phoneNormalized asc)` - `(tenantId asc, emailLower asc)` `(tenantId asc, nameLower asc)` only if exact/prefix search is implemented in a constrained form

Index exemptions: - exe `externalRefs` subfields unless known lookup paths are needed - exempt `tags` unless filtered

`documents/{documentId}`

Purpose: - issued or failed receipts/payment confirmations

Fields:

```
 {
  "documentId": "doc_01H...",
  "tenantId": "t_...",
  "transactionId": "txn_...",
  "providerType": "greeninvoice",
  "providerDocumentId": "gi_123",
  "documentType": "receipt",
  "status": "issued",
  "issueRequestHash": "sha256...",
  "filePath": "gs://bucket/docs/doc_01H.pdf",
  "delivery": {
   "channel": "email",
   "recipient": "john@example.com",
   "deliveredAt": "timestamp",
   "status": "delivered"
  },
  "error": {
   "code": null,
   "message": null
  },
  "createdAt": "timestamp",
```

```
  "updatedAt": "timestamp"
}
```

Recommended composite indexes: `(tenantId asc, status asc, createdAt desc)` - `(tenantId asc, transactionId asc, createdAt desc)` - `(tenantId asc, issueRequestHash asc)`

Index exemptions: - ex `error.message` - exer `filePath` - exer `delivery.recipient` unless needed for support queries

`audit_events/{eventId}`

Purpose: - immutable append-only audit stream

Fields:

```
{
 "eventId": "ae_01H...",
 "tenantId": "t_...",
 "entityType": "transaction",
 "entityId": "txn_...",
 "eventType": "transaction.created",
 "actorType": "system",
 "actorId": "worker-service",
 "correlationId": "corr_123",
 "payload": {
  "status": "succeeded"
 },
 "createdAt": "timestamp"
}
```

Recommended composite indexes: `(tenantId asc, entityType asc, entityId asc, createdAt desc)` `(tenantId asc, eventType asc, createdAt desc)` `(correlationId asc, createdAt desc)` only if correlation tracing is a regular support workflow

Index exemptions: - exempt `payload` from indexing

`sync_runs/{syncRunId}`

Purpose: - operational visibility for polling/webhook execution runs

Fields:

```
{
 "syncRunId": "sr_01H...",
 "tenantId": "t_...",
 "providerConnectionId": "pc_...",
```

```
  "triggerType": "scheduled",
  "status": "succeeded",
  "startedAt": "timestamp",
  "finishedAt": "timestamp",
  "cursorBefore": "opaque",
  "cursorAfter": "opaque2",
  "stats": {
   "fetched": 100,
   "inserted": 95,
   "merged": 3,
   "ignored": 2,
   "failed": 0
  },
  "errorSummary": null
 }
```

Recommended composite indexe`(tenantId asc, providerConnectionId asc, startedAt desc)` - `(tenantId asc, status asc, startedAt desc)`

Index exemptions: - exempt `cursorBefore`, `cursorAfter`, and `errorSummary`

`outbox_events/{eventId}`

Purpose: - optional reliable event publication buffer for Pub/Sub

Fields:

```
 {
  "eventId": "oe_01H...",
  "tenantId": "t_...",
  "topic": "transaction.created",
  "payloadRef": "gs://... or inline small payload",
  "publishStatus": "pending",
  "attemptCount": 0,
  "nextAttemptAt": "timestamp",
  "createdAt": "timestamp",
  "publishedAt": null
 }
```

Recommended composite indexes: - `(publishStatus asc, nextAttemptAt asc)`

This collection is optional for MVP. Use it only if direct Pub/Sub publication inside request/wo proves too brittle.

Approved MVP query catalog

The backend should expose only approved query shapes, not generic arbitrary filtering.

Transactions list: - by tenant, sort by latest - by tenant + providerType - by tenant + stat receiptState - by tenant + customerId - by tenant + exact phoneNormalized - by tenant emailLower - by tenant + date window ordered by occurredAt desc

Documents list: - by tenant + status - by tenant + transactionId

Customers list: - by tenant + exact phoneNormalized - by tenant + exact emailLower - by t prefix-safe nameLower if implemented conservatively

Sync operations: - by tenant + providerConnectionId - by tenant + status

Audit views: - by tenant + entity - by tenant + eventType

Any requested query outside this catalog should be implemented via a new read model, expo or search service rather than by adding unbounded Firestore indexes.

Firestore index management strategy

Manage indexes as code, not manually in the console. Firestore supports both single-field composite indexes, and index definitions can be deployed via Firebase CLI or Terraform. (firebase.google.com)

Recommended approach: - define all composite indexes in source control - define single-field exemptions in source control - review every new UI filter against write cost and query frequ convenience indexes that multiply combinations without clear product value

Practical rule: - every new list filter must name the exact backend query and required ind implementation

Firestore write-path rules

- All transaction ingestion writes must be idempotent.
- Raw payload archive to Cloud Storage should happen before or alongside canonical normalization.
- Dedup reservation must be the only strongly atomic uniqueness guard.
- Avoid writing to many indexed fields in one hot code path because write latency scales with index fanout. Firestore best practices call this out directly. (firebase.google.com)
- Keep batch sizes modest during backfills or replays, and implement retry handling for transient failures. (firebase.google.com)

Firestore pagination strategy

Use cursor-based pagination only.

For example, transactions list `where(tenantId == X)` - optional approved equality filter - `orderBy(occurredAt, desc)` `startAfter(lastOccurredAt, lastDocId)` pattern if needed for stable pagination

Do not use offset pagination because it wastes reads and scales poorly.

Firestore aggregation strategy

For simple dashboard totals, use Firestore aggregation queries only where cardinality and latenc acceptable. Firestore suppo `count`, `sum`, an `average`, but operational dashboards with many segmented totals may still be better served by precomputed counters or exported analytics (firebase.google.com)

MVP recommendation: - allow `count()` for admin lists if needed - avoid building a comple analytics dashboard directly on Firestore queries

Suggested index starter set

Initial composite indexes to provision:

```
transactions:
- tenantId asc, occurredAt desc
- tenantId asc, providerType asc, occurredAt desc
- tenantId asc, status asc, occurredAt desc
- tenantId asc, receiptState asc, occurredAt desc
- tenantId asc, customerId asc, occurredAt desc
- tenantId asc, payer.phoneNormalized asc, occurredAt desc
- tenantId asc, payer.emailLower asc, occurredAt desc

documents:
- tenantId asc, status asc, createdAt desc
- tenantId asc, transactionId asc, createdAt desc
- tenantId asc, issueRequestHash asc

provider_connections:
- tenantId asc, category asc, status asc
- tenantId asc, providerType asc, status asc

sync_runs:
- tenantId asc, providerConnectionId asc, startedAt desc
- tenantId asc, status asc, startedAt desc

audit_events:
- tenantId asc, entityType asc, entityId asc, createdAt desc
- tenantId asc, eventType asc, createdAt desc

auth_sessions:
- userId asc, status asc, createdAt desc

customers:
- tenantId asc, phoneNormalized asc
- tenantId asc, emailLower asc
```

Initial single-field index exemptions to consider:

```
users.passwordHash
provider_connections.syncConfig.cursor
provider_connections.health.lastErrorMessage
transactions.description
transactions.sourceSummary.rawPayloadPath
transactions.tags
documents.filePath
documents.error.message
audit_events.payload
sync_runs.cursorBefore
sync_runs.cursorAfter
sync_runs.errorSummary
```

When Firestore is no longer enough

Introduce supplementary storage/search components if any of these become true: - operators r flexible free-text search across payer, description, and metadata - finance workflows need bro relational joins across transactions, customers, and documents - analytics dashboards require ma segmented aggregates with low latency - complex reconciliation rules need large batch comparisons

At that point, keep Firestore as the operational system of record for the app and add OpenSearch / Algolia for search - BigQuery for reporting and reconciliation - relational store f relational workflows

For the MVP, however, Firestore remains workable as long as the query catalog stays strict are intentionally controlled.

Detailed provider integration architecture

The provider layer is the most change-prone part of the system. It must isolate vendor-spe behavior, webhook formats, pagination rules, credentials(docs.cloud.google.com)ns stable.

Design goals: - keep provider-specific logic out of core transaction/document modules - mak integrations replayable and debuggable - make webhook and polling ingestion converge into canonical pipeline - support partial provider failures without corrupting business data - make the second and third providers mostly configuration plus adapter work

Provider taxonomy

Model providers in two catego `payment_source`: produces transaction events or records - `document_provider`: issues receipts, payment confirmations, or invoice-receipts

Some vendors may later support both categories, but the implementation should still register s adapter capabilities.

Adapter package structure

```
/modules/providers
  /base
    contracts.py
    errors.py
    models.py
    registry.py
    signature.py
    cursor.py
    retry.py
    archival.py
  /payment
    /bit
      adapter.py
      webhook.py
      polling.py
      mapper.py
      schemas.py
    /paybox
      adapter.py
      webhook.py
      polling.py
      mapper.py
      schemas.py
  /documents
    /greeninvoice
      adapter.py
      mapper.py
      schemas.py
    /icount
      adapter.py
      mapper.py
      schemas.py
```

Rules: `adapter.py` exposes the main public integration `webhook.py` contains signature verification and payload parsi `polling.py` contains cursor-based fetch l `mapper.py` converts provider payloads into canonical internal `schemas.py` holds provider-specific request/response validation schemas. - No business workflow should directly parse raw provider outside these modules.

Adapter contracts

Payment provider contract:

```
class PaymentProviderAdapter(Protocol):
  provider_type: str
```

```
  async def validate_credentials(self, credentials: dict) -> ValidationResult: ...
  async def capabilities(self) -> ProviderCapabilities: ...
  async def register_webhook(self, connection: ProviderConnection) -> WebhookRegistrationResult: ...
  async def unregister_webhook(self, connection: ProviderConnection) -> None: ...
  async def parse_webhook(self, headers: dict, body: bytes) -> ProviderWebhookEnvelope: ...
  async def verify_webhook(self, headers: dict, body: bytes, secret: str | None) -> VerificationResult: ...
  async def fetch_incremental(self, connection: ProviderConnection, cursor: ProviderCursor | None) -> FetchPageResult: ...
  async def normalize_event(self, envelope: ProviderWebhookEnvelope | ProviderRecord) -> CanonicalTransactionInput: ...
  async def fetch_transaction_detail(self, connection: ProviderConnection, external_id: str) -> ProviderRecord | None: ...
```

Document provider contract:

```
class DocumentProviderAdapter(Protocol):
  provider_type: str

  async def validate_credentials(self, credentials: dict) -> ValidationResult: ...
  async def capabilities(self) -> DocumentCapabilities: ...
  async def issue_document(self, request: IssueDocumentRequest) -> IssueDocumentResult: ...
  async def get_document(self, connection: ProviderConnection, external_id: str) -> DocumentFetchResult: ...
  async def deliver_document(self, request: DeliverDocumentRequest) -> DeliveryResult: ...
```

Shared provider capability model

Each adapter must declare its capabilities so the application can choose the right ingestion strategy.

Example capability object:

```
{
 "supportsWebhooks": true,
 "supportsPolling": true,
 "supportsCursorPagination": true,
 "supportsReplayByDate": true,
 "supportsDocumentDelivery": false,
 "rateLimit": {
  "requestsPerMinute": 60,
  "burst": 10
```

```
  }
}
```

This allows the backend to avoid hardcoding provider branches in orchestration logic.

Provider connection lifecycle

Connection states: - `draft` - `validating` - `active` - `error` - `reauth_required` - `disabled`

Lifecycle: 1. Admin enters credentials or OAuth-derived tokens. 2. Backend stores secret mater in Secret Manager and writes only a reference into Firestore. 3. Adapter validates credentials. If supported, adapter registers webhook. 5. Initial sync task is scheduled. 6. Connection beco `active` only after a successful validation and first sync handshake.

Secret storage model

Never store API keys or provider access tokens directly in Firestore.

Use Secret Manager with one secret per provider connection or credential bundle.

Recommended secret naming schem `provider-{tenantId}-{connectionId}` `webhook-{providerType}-{connectionId}` if provider requires separate webhook secret handling

Store in Firestore c `credentialsRef` - token expiry metadata if needed for refresh workflows non-sensitive connection metadata

Webhook architecture

All provider webhooks should arrive at the API service through a pro `POST /v1/c` route: - `providers/{providerType}/webhook`

Webhook processing stages: 1. Resolve provider adapter from route. 2. Load provider connecti candidates if needed. 3. Verify signature/authenticity. 4. Persist raw request body and selecte headers to Cloud Storage. 5. Parse event envelope. 6. Write a lightweight intake audit ev 7. Enqueue Cloud Task for canonical processing unless the provider event is tiny and safe inline. 8. Return HTTP success quickly after authenticity and archival are complete.

Webhook endpoint rules: - keep synchronous response work minimal - never call downstrea document issuance synchronously from webhook request path - never discard failed signature att silently; record security audit events - preserve original request bytes for replay/debugging

Webhook raw payload archival format

Recommended Cloud Storage path:

```
raw/provider={providerType}/tenant={tenantId_or_unknown}/date=YYYY-MM-DD/
source=webhook/connection={connectionId_or_unknown}/
event={eventId_or_uuid}.json
```

Stored payload envelope:

```
{
 "receivedAt": "timestamp",
 "providerType": "bit",
 "connectionId": "pc_...",
 "headers": {
  "content-type": "application/json",
  "x-signature": "..."
 },
 "query": {},
 "bodyBase64": "...",
 "contentHash": "sha256..."
}
```

This makes exact replay possible even when provider parsing code changes later.

Webhook verification model

Implement a provider-specific verification strategy abstraction:

```
class WebhookVerifier(Protocol):
  async def verify(self, headers: dict, body: bytes, secret: str | None) ->
VerificationResult: ...
```

Verification outcomes: `verified` `unverified_allowed` for providers that do not sign but can still be restricted by route secret or source controls - `failed` - `not_supported`

For unsigned providers, compensate with: - secret callback URL path component if vendor all source IP allowlisting only as a secondary control if vendor documents fixed ranges - aggres limiting and anomaly audit logging

Polling architecture

Polling exists for providers that: - do not support webhooks - have unreliable webhooks - periodic reconciliation even when webhooks exist

Polling flow: 1. Cloud Scheduler triggers a tenant/provider sync start endpoint. 2. API identi active due connections. 3. API enqueues one Cloud Task per connection. .4 Worker load connection metadata and secrets. 5. Adapter fetches one page incrementally. 6. Every returne record is archived and normalized. 7. Dedup pipeline writes canonical transactions. 8. If ther

another page, worker enqueues the next page task with updated cursor. 9. When finishe `sync_runs` is closed and connection health is updated.

Polling rule: - process one page per task invocation for predictability and retry safety

Cursor model

Cursors should be opaque to the orchestration layer.

Shared cursor wrapper:

```
{
 "version": 1,
 "mode": "provider_cursor",
 "value": "opaque-token",
 "watermark": "2026-03-10T10:00:00Z",
 "page": 3
}
```

Supported cursor strategies: - opaque provider token - last seen timestamp - last seen exte date-window replay checkpoints

Cursor rules: - store only the last committed successful cursor in `provider_connections.syncConfig.cursor` - within an active run, keep tentative page progress inside th `sync_runs` document or task payload - advance the durable cursor only after the p been archived and canonically processed successfully

Unified intake pipeline

Webhook and polling paths must converge into the same canonical intake service.

Canonical pipeline stages: 1. archive raw payload 2. parse provider envelope/record 3. normalize to canonical transaction input 4. derive dedup keys 5. reserve dedup key 6. cre or merge transaction 7. emit audit event 8. enqueue side effects

This prevents drift where webhook transactions and polled transactions behave differently.

Provider retry policy

Retries must be different for three failure types:

1. **Upstream provider call failures**

Examples: - timeout - connection reset - HTTP 429 - provider 5xx

Policy: - retry via Cloud Tasks with exponential backoff - preserve correlation ID across atte update `provider_connections.health` - if repeated threshold exceeded, mark `error` ection and notify operator

2 . **Canonical processing failures**

Examples: - schema mismatch - unexpected required field absence - serialization bug

Policy: - do not blindly retry forever - write the raw payload path, error classification, an snapshot - route to dead-letter review after a bounded retry count - allow operator replay a fix or rules update

3 . **Downstream document issuance failures**

Examples: - invoicing provider downtime - validation error in issuance payload

Policy: - retry only if error is classified recoverable - terminal validation failures should stop actionable remediation

Provider error taxonomy

Every adapter must translate vendor-specific errors into a shared internal taxonomy.

Recommended taxonomy: - `auth.invalid_credentials` - `auth.expired_token` - `auth.insufficient_scope` `network.timeout` `network.unreachable` `rate_limited` - `upstream.unavailable` - `upstream.invalid_response` - `webhook.signature_failed` - `mapping.unsupported_payload` - `mapping.missing_required_field` - `state.conflict_duplicate` - `state.not_found` - `document.validation_failed` - `document.delivery_failed` - `internal.unexpected`

Error object shape:

```
{
 "code": "rate_limited",
 "retryable": true,
 "providerCode": "429",
 "message": "Upstream provider rate limit exceeded",
 "details": {
  "retryAfterSeconds": 60
 }
}
```

This taxonomy drives retries, alerts, and admin messaging.

Connection health model

Update `provider_connections.health` after each validation, sync, and webhook verification event.

Recommended fields: - `lastSuccessAt` - `lastFailureAt` - `lastErrorCode` - `lastErrorMessage` - `consecutiveFailureCount` - `webhookLastReceivedAt` - `webhookLastVerifiedAt` - `latencyP95Ms` (optional later)

This powers the provider status screen in mobile/admin.

Replay and debugging model

Every provider workflow must be replayable from stored raw artifacts.

Replay modes: - replay single webhook payload - replay one sync run - replay one provide external ID - replay failed dead-letter items after code fix

Replay safety rules: - replay always goes through the same dedup reservation logic - replay bypass audit logging - replay preserves linkage `correlationId` plus `replayOf` reference

Rate limiting and concurrency

Each provider connection should have its own effective concurrency budget to avoid account-l throttling.

Recommended rules: - at most one active polling task per connection - separate Cloud Tasks queue routing `sync` an `documents` - adapter declares recommended request pacing - task payload carries retry count and backoff hints

Provider domain events

After canonical processing, publish small domain events s `transaction.created` - `transaction.updated` - `document.issue_requested` - `document.issued` - `provider.connection_failed`

These should carry compact references, not full payloads.

Example event:

```
{
 "eventType": "transaction.created",
 "tenantId": "t_...",
 "transactionId": "txn_...",
 "providerType": "bit",
 "occurredAt": "timestamp",
 "correlationId": "corr_123"
}
```

Internal service boundaries for provider orchestration

Recommended internal service `ProviderRegistryService` `ProviderCredentialService` - `WebhookIntakeService` - `SyncOrchestratorService` - `CanonicalIngestionService` - `DocumentIssuanceService` - `ReplayService` - `ProviderHealthService`

This keeps orchestration logic testable without coupling directly to HTTP handlers.

Operational dashboards for integrations

Admin web should expose: - connection status and last validation result - last webhook rece last verified timestamp - last sync run and sync statistics - recent provider errors by nor taxonomy - replay/retry actions for privileged operators

Acceptance criteria for each new provider

A provider integration is not complete until it has: - credential validation - webhook verifica documented polling fallback - incremental sync with cursor persistence - canonical mapping tests real sample payloads - dedup safety coverage for webhook + polling overlap - raw payload normalized error mapping - replay of stored payloads in non-production - admin visibility for health

Idempotency and deduplication algorithm

The most important correctness mechanism in the MVP is duplicate-safe ingestion.

Dedup priority order: 1. provider event ID 2. provider transaction ID 3. deterministic fingerprint

Fingerprint input example: - tenantId - providerType - normalized amount - normalized curren occurredAt rounded to safe precision if needed - payer normalized phone/email if present - settlement/reference token if present

Algorithm: 1. Build candidate keys. 2. Attempt atomic i `transaction_dedup` for strongest available key. 3. If key already exists, fetch referenced transaction and decide merg op. 4. If inserted, create canonical transaction. 5. Insert weaker dedup aliases pointing to same transaction. 6. Write audit event. 7. Publ `transaction.created` or `transaction.updated` event.

Receipt/document issuance flow

Document issuance must be retryable and idempotent.

Flow: 1. User or policy marks a transaction as requiring a document. 2. API builds a n `IssueDocumentRequest`. 3. Request hash is computed from tenant, transaction, provider, document type, and payload. 4. Cloud Task is enqueued with the request hash as a dedup Worker checks for an existing `documents` record with the same hash. 6. If found, return existing result. 7. Otherwise call the document provider adapter. 8. Store result, archive file Cloud Storage if returned, update tran `receiptState`, and write audit event. 9. On recoverable failure, retry with exponential backoff. 10. On terminal failure, mark document `failed` and notify the tenant.

Sync orchestration

There are two sync entry points: - Webhook-first providers - Polling-first providers

Webhook path: - Verify signature. - Store raw payload. - Normalize event. - Upsert canonical - Enqueue heavy side effects asynchronously.

Polling path: - Cloud Scheduler `start_sync` endpoint per active connection. - API enqueues Cloud Task for each connection. - Worker fetches incremental pages using stored cursor. - Ea item goes through the same normalization/dedup pipeline. - Cursor is advanced only after suc page processing.

Detailed API contract and endpoint payloads

Expose versioned REST APIs `/v1` er for public app/admin usa `/internal` for service-to-service task/event execution. FastAPI maps naturally to Pydantic request/response models and Ope generation, which is useful here because mobile, admin web, and contractor teams need one contract surface. (fastapi.tiangolo.com) (fastapi.tiangolo.com)

API design rules: - Every endpoint has explicit request and response models. - Every mutatio the canonical server state after the write. - All list endpoints use cursor pagination, not pagination; this aligns with Firestore's cursor-based pagination (firebase.google.com) - Public APIs require bearer access tokens. - Internal APIs require Cloud Run service-to-service authentic and should not be publicly callable. Cloud Run supports authenticated service-to-service requests ID tokens and IAM. (docs.cloud.google.com) - Retryable async work is represented as Cloud Tasks HT callbacks, which Cloud Tasks supports directly for HTTP endpoints. (docs.cloud.google.com) - Use idempotency keys for user-triggered mutations that may be repeated by flaky mobile networks.

API envelope conventions

Standard success response:

```
{
 "data": {},
 "meta": {
  "requestId": "req_123"
 }
}
```

Standard error response:

```
{
 "error": {
  "code": "validation.failed",
  "message": "One or more fields are invalid.",
  "details": {
   "fieldErrors": {
    "email": ["Invalid email format"]
   }
  }
 },
 "meta": {
```

```
  "requestId": "req_123"
 }
}
```

Error response rule `code` is stable for client `message` - is human-readable but not relied on programmatically. `details` is optional and endpoint-specific. - Internal provider exceptions are translated before leaving the API boundary.

Common headers

Public API header `Authorization: Bearer <access_token>` `X-Tenant-Id: <tenantId>` when tenant context is required and not taken from token/session state - `Idempotency-Key: <uuid>` for idempotent mutati `X-Request-Id: <uuid>` optional client correlation header

Internal API headers: - authenticated Cloud Run identity `X-Internal-Request-Id` - `X-Task-Name` / `X-CloudTasks-TaskName` if invoked by Cloud Tasks

Pagination contract

All list endpoints should return:

```
{
 "data": [ ... ],
 "meta": {
  "nextCursor": "opaque_cursor",
  "hasMore": true
 }
}
```

Cursor content should be opaque to the client, but can encode: - sort timestamp - last d applied filters checksum

The backend must reject a cursor if it does not match the current filter set.

Idempotency contract

Idempotency is required for these endpoints at minimum: - login optional but not necessary connection create/update - issue document - manual sync trigger - password reset completion but recommended

Idempotency rules: - client `Idempotency-Key` - backend stores short-lived operation record keyed by tenant + user + endpoint + key - if the same key and logically equivalent paylo return the prior result - if the same key arrives with a different payload, return `409 conflict`

Suggested collection: - `idempotency_keys/{scope_hash}`

Suggested fields:

```
{
 "scope": "tenant:user:endpoint",
 "key": "uuid",
 "requestHash": "sha256...",
 "responseSnapshot": {},
 "statusCode": 200,
 "createdAt": "timestamp",
 "expiresAt": "timestamp"
}
```

Auth API contracts

`POST /v1/auth/register`

Request:

```
{
 "email": "owner@example.com",
 "password": "strong-password",
 "displayName": "Jane Doe",
 "tenant": {
  "name": "Acme Payments",
  "countryCode": "IL",
  "timezone": "Asia/Jerusalem",
  "currency": "ILS"
 }
}
```

Response:

```
{
 "data": {
  "user": {
   "userId": "u_123",
   "email": "owner@example.com",
   "displayName": "Jane Doe"
  },
  "tenant": {
   "tenantId": "t_123",
   "name": "Acme Payments",
   "role": "owner"
  },
  "session": {
   "accessToken": "...",
   "expiresInSec": 900,
   "refreshToken": "..."
```

```
    }
  },
  "meta": {
    "requestId": "req_123"
  }
}
```

Validation rules: - normalize email to lowercase - password policy enforced server-side - if em exists, return `409 auth.email_already_exists`

`POST /v1/auth/login`

Request:

```
{
  "email": "owner@example.com",
  "password": "strong-password",
  "device": {
    "platform": "ios",
    "appVersion": "1.0.0",
    "deviceLabel": "iPhone 15"
  }
}
```

Response: - sa `session` structure as register - `tenants` array if the user belongs to multiple tenants later

Failure codes: `401 auth.invalid_credentials` `423 auth.account_locked` `429 auth.rate_limited`

`POST /v1/auth/refresh`

Request:

```
{
  "refreshToken": "..."
}
```

Response:

```
{
  "data": {
    "accessToken": "...",
    "expiresInSec": 900,
    "refreshToken": "rotated..."
```

```
  },
  "meta": {
   "requestId": "req_123"
  }
}
```

Rules: - rotate refresh token on every successful refresh - revoke old token version immediately

`POST /v1/auth/logout`

Request:

```
{
 "refreshToken": "..."
}
```

Response:

```
{
 "data": {
  "loggedOut": true
 },
 "meta": {
  "requestId": "req_123"
 }
}
```

`POST /v1/auth/forgot-password`

Request:

```
{
 "email": "owner@example.com"
}
```

Response:

```
{
 "data": {
  "accepted": true
 },
 "meta": {
  "requestId": "req_123"
```

```
  }
}
```

Rule: - always return success-like response to avoid account enumeration

`POST /v1/auth/reset-password`

Request:

```
{
 "token": "reset_token",
 "newPassword": "new-strong-password"
}
```

Response:

```
{
 "data": {
  "passwordReset": true,
  "sessionsRevoked": true
 },
 "meta": {
  "requestId": "req_123"
 }
}
```

`GET /v1/me`

Response:

```
{
 "data": {
  "userId": "u_123",
  "email": "owner@example.com",
  "displayName": "Jane Doe",
  "activeTenant": {
   "tenantId": "t_123",
   "name": "Acme Payments",
   "role": "owner"
  }
 },
 "meta": {
  "requestId": "req_123"
```

```
  }
}
```

Tenant and provider connection API contracts

`GET /v1/tenants/{tenantId}/provider-connections`

Response item:

```
{
 "connectionId": "pc_123",
 "providerType": "bit",
 "category": "payment_source",
 "displayName": "Bit Main Account",
 "status": "active",
 "health": {
  "lastSuccessAt": "timestamp",
  "lastErrorCode": null,
  "consecutiveFailureCount": 0,
  "webhookLastReceivedAt": "timestamp"
 },
 "syncConfig": {
  "mode": "webhook_plus_poll",
  "lastSyncAt": "timestamp",
  "nextSyncAt": "timestamp"
 }
}
```

`POST /v1/tenants/{tenantId}/provider-connections`

Request:

```
{
 "providerType": "bit",
 "category": "payment_source",
 "displayName": "Bit Main Account",
 "credentials": {
  "apiKey": "secret",
  "accountId": "123"
 },
 "syncConfig": {
  "mode": "webhook_plus_poll"
 }
}
```

Response:

```
{
 "data": {
  "connectionId": "pc_123",
  "status": "validating",
  "providerType": "bit",
  "category": "payment_source"
 },
 "meta": {
  "requestId": "req_123"
 }
}
```

Behavior: - credentials are written to Secret Manager, not Firestore - response does not echo enqueue validation task immediately

`PATCH /v1/tenants/{tenantId}/provider-connections/{connectionId}`

Request supports partial update `displayName` - credential replacement - enable/disable state - sync mode where supported

`POST /v1/tenants/{tenantId}/provider-connections/{connectionId}/validate`

Response:

```
{
 "data": {
  "queued": true,
  "connectionId": "pc_123"
 },
 "meta": {
  "requestId": "req_123"
 }
}
```

`POST /v1/tenants/{tenantId}/provider-connections/{connectionId}/sync`

Request:

```
{
 "mode": "incremental",
 "reason": "manual"
}
```

Response:

```
{
 "data": {
  "queued": true,
  "syncRunId": "sr_123"
 },
 "meta": {
  "requestId": "req_123"
 }
}
```

Transactions API contracts

`GET /v1/tenants/{tenantId}/transactions`

Approved query params for MV `cursor` · `limit` default `20`, max `100` - `providerType` `status` `receiptState` `customerId` `phoneNormalized` `emailLower` - `dateFrom` - `dateTo`

Response item:

```
{
 "transactionId": "txn_123",
 "providerType": "bit",
 "status": "succeeded",
 "amount": {
  "valueMinor": 1050,
  "currency": "ILS"
 },
 "occurredAt": "timestamp",
 "payer": {
  "name": "John Doe",
  "phone": "+972...",
  "email": "john@example.com"
 },
 "receiptState": "issued",
 "documentId": "doc_123",
 "tags": []
}
```

Query rules: - only one primary equality filter family in MVP plus optional date window unsupported combinations with `400 query.unsupported_filter_combination` - sort by `occurredAt desc`

`GET /v1/tenants/{tenantId}/transactions/{transactionId}`

Response:

```
{
 "data": {
  "transactionId": "txn_123",
  "tenantId": "t_123",
  "providerType": "bit",
  "providerConnectionId": "pc_123",
  "providerTransactionId": "ext_123",
  "status": "succeeded",
  "amount": {
   "valueMinor": 1050,
   "currency": "ILS"
  },
  "occurredAt": "timestamp",
  "capturedAt": "timestamp",
  "payer": {
   "name": "John Doe",
   "phone": "+972...",
   "email": "john@example.com"
  },
  "customerId": "cust_123",
  "description": "Order #1234",
  "tags": ["vip"],
  "receiptState": "issued",
  "documentId": "doc_123",
  "sourceSummary": {
   "channel": "webhook",
   "settlementRef": "set_123"
  },
  "auditPreview": [
   {
    "eventType": "transaction.created",
    "createdAt": "timestamp"
   }
  ]
 },
 "meta": {
  "requestId": "req_123"
 }
}
```

`POST /v1/tenants/{tenantId}/transactions/{transactionId}/issue-document`

Headers: - `Idempotency-Key` required

Request:

```
{
 "documentType": "receipt",
```

```
  "providerConnectionId": "pc_doc_123",
  "delivery": {
   "channel": "email",
   "recipient": "john@example.com"
  }
 }
```

Response:

```
{
 "data": {
  "queued": true,
  "documentRequest": {
   "transactionId": "txn_123",
   "documentType": "receipt",
   "status": "queued"
  }
 },
 "meta": {
  "requestId": "req_123"
 }
}
```

Behavior: - compute request hash server-side - return existing successful document result if lo identical request was already completed - return `409 state.document_already_exists_conflict` for conflicting issuance request

Documents API contracts

`GET /v1/tenants/{tenantId}/documents/{documentId}`

Response:

```
{
 "data": {
  "documentId": "doc_123",
  "transactionId": "txn_123",
  "providerType": "greeninvoice",
  "documentType": "receipt",
  "status": "issued",
  "fileUrl": null,
  "delivery": {
   "channel": "email",
   "recipient": "john@example.com",
   "status": "delivered",
   "deliveredAt": "timestamp"
  },
```

```
  "createdAt": "timestamp"
 },
 "meta": {
  "requestId": "req_123"
 }
}
```

Rule: - prefer short-lived signed download URLs generated on demand instead of storing pub URLs

Customers API contracts

`GET /v1/tenants/{tenantId}/customers`

Supported query params `phoneNormalized` `emailLower` `namePrefix` optional only if implemented with controlled semantics - `cursor` - `limit`

`GET /v1/tenants/{tenantId}/customers/{customerId}`

- returns customer profile plus recent transactions summary

Audit and sync visibility API contracts

`GET /v1/tenants/{tenantId}/sync-runs`

Supported params: - `providerConnectionId` - `status` - `cursor` - `limit`

`GET /v1/tenants/{tenantId}/audit-events`

Supported params: - `entityType` - `entityId` - `eventType` - `cursor` - `limit`

These endpoints are primarily for admin web and support tooling, not mobile-first surfaces.

Webhook endpoint contract

`POST /v1/providers/{providerType}/webhook`

Request: - raw provider-specific payload - provider-specific headers/signature

Response on accepted intake:

```
{
 "data": {
  "accepted": true
 },
 "meta": {
  "requestId": "req_123"
```

```
  }
}
```

Rules: - respond quickly after authenticity verification and archival - do not leak detailed failu to unauthenticated callers - on signature failure, return provider-compatible error code while rec audit details internally

Internal task API contracts

Internal endpoints should be private Cloud Run services or private routes callable only through to-service authentication. Cloud Run documents IAM-backed authenticated service invocations for th purpose. (docs.cloud.google.com)

`POST /internal/tasks/process-sync-page`

Request:

```
{
 "syncRunId": "sr_123",
 "tenantId": "t_123",
 "providerConnectionId": "pc_123",
 "cursor": {
  "version": 1,
  "mode": "provider_cursor",
  "value": "opaque_cursor"
 },
 "attempt": 1,
 "correlationId": "corr_123"
}
```

Response:

```
{
 "data": {
  "processed": true,
  "nextPageQueued": true,
  "stats": {
   "fetched": 100,
   "inserted": 95,
   "merged": 3,
   "ignored": 2,
   "failed": 0
  }
 },
 "meta": {
  "requestId": "req_123"
```

```
  }
}
```

`POST /internal/tasks/issue-document`

Request:

```
{
 "tenantId": "t_123",
 "transactionId": "txn_123",
 "providerConnectionId": "pc_doc_123",
 "documentType": "receipt",
 "delivery": {
  "channel": "email",
  "recipient": "john@example.com"
 },
 "requestHash": "sha256...",
 "attempt": 1,
 "correlationId": "corr_123"
}
```

Response:

```
{
 "data": {
  "processed": true,
  "documentId": "doc_123",
  "status": "issued"
 },
 "meta": {
  "requestId": "req_123"
 }
}
```

`POST /internal/tasks/validate-provider-connection`

Request:

```
{
 "tenantId": "t_123",
 "providerConnectionId": "pc_123",
 "attempt": 1,
 "correlationId": "corr_123"
}
```

`POST /internal/events/transaction-created`

Request:

```
{
 "eventType": "transaction.created",
 "tenantId": "t_123",
 "transactionId": "txn_123",
 "providerType": "bit",
 "correlationId": "corr_123"
}
```

Use this only when you want explicit internal HTTP event handlers; otherwise direct Pub/S subscription handlers are cleaner.

Cloud Tasks payload and naming rules

Cloud Tasks supports HTTP target tasks and queue-level control over retries and dispatch. (docs.cloud.google.com) (docs.cloud.google.com)

Recommended task payload rules: - use JSON payloads `correlationId`, `attempt`, and resource identifiers - keep payloads small; fetch heavy state from Firestore or Storage i handler - when possible, use deterministic task names for dedup-sensitive operations such a document issuance

Recommended task names: `sync-{syncRunId}-{pageHash}` - `issue-doc-{tenantId}-{transactionId}-{requestHash}` - `validate-conn-{providerConnectionId}`

Response model policy

FastAPI response models should be used consistently so the OpenAPI document filters and va output shapes. (fastapi.tiangolo.com)

Policy: - define request/response Pydantic models per endpoint - never return raw Firestore doc directly - map internal entities to explicit API DTOs - redact i `credentialsRef` ch as `passwordHash`, `rawPayloadPath`, and internal error traces

API versioning policy

Use URI versioning: - `/v1/...`

Rules: - additive fields are allowed within the same version - breaking field removals or changes requir `/v2` - internal endpoints may evolve faster but should still be versioned once workers/consumers exist

OpenAPI and client generation

FastAPI can automatically generate OpenAPI docs from typed request/response models and metad which is useful for client generation and contractor handoff. (fastapi.tiangolo.com)

Recommendation: - publish OpenAPI from staging - generate a typed admin-web client from OpenAPI spec - generate mobile DTOs or at least contract tests from the same spec

API acceptance criteria

The API layer is ready for implementation when: - every public endpoint has a typed reque contract - every list endpoint has one of the approved Firestore-backed query shapes - mu endpoints define idempotency behavior - internal task endpoints are private and authenticated sensitive fields are excluded from response models - OpenAPI is generated and reviewed by r admin teams

Security model

- Authentication handled by the application in MVP using email/password.
- Passwords stored only as strong s(cheatsheetseries.owasp.org)I issues and validates signed access tokens and refresh tokens.
- Authorization enforced in the backend, never trusted from client claims alone.
- Identity and authorization are separated: authentication proves user identity, while tenant membership and role permissions remain in application data.
- Provider credentials stored in Secret Manager.
- Webhook signatures verified per provider.
- PII fields minimized and access-logged.
- Admin/internal task endpoints protected by service-to-service identity.

Detailed auth and user/account architecture

The MVP should use application-managed authentication with email/password because it is the cheapest way to launch while still being production-credible. The design must preserve a stable `userId` so later login methods such as Google OAuth or phone OTP can be linked to account rather than creating parallel user identities.

Design goals: - keep sign-in simple for MVP - separate authentication from tenant/business authorization - support mobile and admin web with the same backend auth system - suppor revocation and refresh-token rotation - make later Google OAuth and phone OTP additive, disruptive

Identity model

Use one canonical application user record per person.

Identity concepts: `userId`: stable internal identifier used everywhere in bus `auth` data - `method`: password, Google OAuth, or phor `membership`: the user's role inside a tenant - `session`: one device/browser login context tied to refresh-token state

This separation ensures future auth expansion does not require migrating transaction, audit, membership records.

Authentication flow for MVP

MVP sign-in method: - email + password

Recommended flow: 1. User registers with email and password. 2. Backend normalizes email lowercase for uniqueness checks. 3. Password is hashed server-side with a memory-hard adapt password hasher. 4. Backend creates the user record and initial tenant or membership. 5 login, backend verifies password hash. 6. Backend issues: - short-lived access token - longer- refresh token 7. Refresh token is stored hashed in Firestore session state. 8. Mobile/admin the access token for API calls and refresh when needed.

OWASP currently recommends memory-hard password hashing such as Argon id, with concrete minimum guidance, and advises against fast general-purpose hashing for password storage. (cheatsheetseries.owasp.org)

Password hashing recommendation

Recommended algorithm: - Argon 2 id

Fallback only if required by library/platform constraints: - bcrypt with an appropriate work factor

Implementation rules: - never store plaintext passwords - never store reversible encrypted passw use a library that automatically salts hashes - keep algorithm parameters configurable for futur - include `passwordHashVersion` field so old hashes can be rehashed on next login wher parameters change

Suggested user auth fie `passwordHash` `passwordHashVersion` `passwordChangedAt` - `failedLoginCount` - `lockedUntil`

Token model

Use bearer tokens for API access.

Recommended token split: - access token: 15 minutes - refresh token: 30 days, rotatable

Access token contents:

```
{
 "sub": "u_01H...",
 "sid": "s_01H...",
 "activeTenantId": "t_...",
 "type": "access",
 "iat": 1710000000,
```

```
  "exp": 1710000900
}
```

Refresh token contents: - may be opaque random string or signed token - recommended app MVP: opaque random token returned once, with only its hash stored server-side

Why this split: - access tokens stay short-lived and lightweight - refresh tokens can be rev device/session - server keeps control without doing a database read on every access-token val unless business context requires it

FastAPI's current security guidance supports a standard bearer-token pattern for API authenticati (fastapi.tiangolo.com)

Session model

Use a dedicated session record per device/browser login.

`auth_sessions/{sessionId}` fields:

```
{
 "sessionId": "s_01H...",
 "userId": "u_...",
 "tenantId": "t_...",
 "refreshTokenHash": "sha256...",
 "refreshTokenVersion": 1,
 "deviceInfo": {
  "platform": "ios",
  "appVersion": "1.0.0",
  "deviceLabel": "iPhone 15"
 },
 "ipAddressLastSeen": "1.2.3.4",
 "userAgent": "...",
 "status": "active",
 "createdAt": "timestamp",
 "lastSeenAt": "timestamp",
 "expiresAt": "timestamp",
 "revokedAt": null,
 "revokedReason": null
}
```

Session rules: - one login creates one session - refresh rotates the refresh token and update hash - logout revokes the current session only - admin/security action may revoke all sessions - password change revokes all sessions except optionally the current one

Recommended auth endpoints

```
POST /v1/auth/register
POST /v1/auth/login
POST /v1/auth/refresh
POST /v1/auth/logout
POST /v1/auth/logout-all
POST /v1/auth/forgot-password
POST /v1/auth/reset-password
GET  /v1/me
GET  /v1/me/sessions
DELETE /v1/me/sessions/{sessionId}
```

Registration policy

For MVP speed, keep registration simple.

Recommended policy: - email required - password required - email uniqueness enforced on nor lowercase email - optional display name - optional phone field stored but not used for lo registrant for a new business becomes tenant owner

Optional but recommended even in MVP: - email verification before certain sensitive actions

If email verification is deferred, compensate with: - stricter password reset verification - rate l registration and login - admin controls for suspicious accounts

Password reset flow

Use email-based password reset.

Flow: 1. User requests reset by email. 2. Backend creates a short-lived one-time reset to 3. Store only a hash of the reset token in Firestore. 4. Send reset URL by email. 5. new password with token. 6. Backend verifies token hash, expiry, and unused status. 7. Ba writes new password hash and revokes all sessions.

Suggested collection: - `password_reset_tokens/{tokenId}`

Fields:

```
{
 "tokenId": "prt_01H...",
 "userId": "u_...",
 "emailLower": "owner@example.com",
 "tokenHash": "sha256...",
 "status": "active",
 "createdAt": "timestamp",
 "expiresAt": "timestamp",
```

```
  "usedAt": null
}
```

Indexing: - direct doc lookup by I`(emailLower, status, createdAt desc)` if support tooling needs it

Email verification model

Recommended approach: - do not block MVP launch on verification - support optional verificati flow in phase 1.5 or phase 2 - mark user records `emailVerified` and `emailVerifiedAt`

Sensitive actions that can later require verified email: - changing password - adding OAuth b changing business/legal settings - exporting sensitive data

Login abuse protections

Implement basic protections from day one: - rate limit login endpoint by IP and by normal increment `failedLoginCount` - apply temporary lockout after repeated failures - add audit eve for lockout and repeated login failures - use generic login error messages to avoid acc enumeration

Suggested lockout example: - 5 failed attempts within rolling window -> 15 minute lock reset failed count on successful login

Tenant authorization model

Authentication proves who the user is. Authorization decides what they can do in a tenant.

Access control flow per request: 1. Validate access token. 2. Load user membership for `activeTenantId`. 3. Check role/permissions `tenant_memberships`. 4. Apply endpoint policy.

Do not embed full permission sets permanently inside tokens because membership may change token expiry. Tokens may i `activeTenantId` and basic identity, but the backend should still resolve current authorization state from application data.

Multi-tenant session behavior

A user may belong to multiple tenants later.

Recommended MVP behavior: - one active tenant context per session - allow tenant switch e multiple memberships exist - on tenant switch, issue a new access token with the ne `activeTenantId` - keep membership checks server-side

This avoids putting an entire tenant list in each token.

Google OAuth future path

Do not implement in MVP, but design for linking later.

When added: - Google login should resolve by verified email first if safe - otherwise create/binding explicitly after re-authentication - store provider linkage in the existing user record `user_auth_identities`

Recommended future collection if multiple auth methods `user_auth_identities/{identityId}`

Example fields:

```
{
 "identityId": "ua_01H...",
 "userId": "u_...",
 "provider": "google",
 "providerSubject": "google-oauth-subject",
 "email": "owner@example.com",
 "linkedAt": "timestamp"
}
```

Phone OTP future path

Phone OTP should be additive, not a replacement of the user model.

Migration strategy: 1. ke`userId` stable 2. add phone verification fields to existing users 3. add a phone identity binding once the phone is verified 4. allow sign-in by phone uniqueness and recovery policy are defined

Suggested future user fields: `phone` - `phoneNormalized` - `phoneVerified` - `phoneVerifiedAt` - `preferredLoginMethod`

Suggested future OTP collection: - `otp_challenges/{challengeId}`

Fields:

```
{
 "challengeId": "otp_01H...",
 "userId": "u_...",
 "phoneNormalized": "9725...",
 "channel": "sms",
 "purpose": "login",
 "codeHash": "sha256...",
 "attemptCount": 0,
 "maxAttempts": 5,
 "createdAt": "timestamp",
```

```
  "expiresAt": "timestamp",
  "verifiedAt": null,
  "status": "active"
}
```

Account recovery policy

MVP recovery path: - email-based password reset only

Future recovery paths: - verified Google identity - verified phone OTP - admin-assisted recove business owners in exceptional cases

Recovery rules: - recovery event revokes existing sessions - audit all recovery events - requir authentication for high-risk account changes

Audit events for auth

Track these auth/security events: `auth.registered` - `auth.login_succeeded` - `auth.login_failed` - `auth.locked` - `auth.refresh_succeeded` - `auth.logout` - `auth.logout_all` `auth.password_reset_requested` `auth.password_reset_completed` - `auth.email_verified` - `auth.identity_linked`

Recommended libraries and secrets

Python libraries: - password hashing library with Argon 2id support - PyJWT or python-jose for token signing - cryptographically secure random token generation for refresh/reset tokens

Secrets to keep in Secret Manager: - JWT signing key(s) - email provider API credentials - client secrets

Google recommends storing sensitive values for Cloud Run services in Secret Manager rather directly in environment configuration. (docs.cloud.google.com)

Auth acceptance criteria

The auth subsystem is MVP-complete when it supports: - registration with email/password - lo refresh flow - logout current session and logout all sessions - password reset by email - authorization on protected endpoints - session revocation after password change - rate limiting lockout for repeated failures - audit logging for auth-sensitive actions

Mobile architecture

Flutter app should use: - feature-based modules - repository pattern over API clients - local recent transactions and pending actions - push notification handling for failures and action-requ events - optimistic UI only for safe actions; authoritative state always comes from backend

Recommended screens for MVP: - login with email/password - tenant switcher (if relevant) - t list - transaction details - issue receipt/document action - provider connection status - notifica inbox - settings/profile

Admin web responsibilities

The admin web is intentionally small and should handle: - tenant onboarding/configuration - p connection setup and re-auth - branding settings - support review of failed syncs/documents - visibility - exports

Suggested codebase structure

```
/backend
  /app
    /api
    /modules
      /auth
      /tenants
      /providers
        /payment
        /documents
      /transactions
      /customers
      /documents
      /notifications
      /audit
      /reporting
    /infra
      /firestore
      /gcs
      /tasks
      /pubsub
      /secrets
      /auth
    /workers
    /schemas
    /config
/mobile
  /lib
    /features
    /core
    /data
/admin-web
  /src
```

Observability, reliability, and SRE model

The architecture should be designed so that operators can quickly answer five questions: - I healthy right now? - Are provider webhooks (docs.cloud.google.com)ce jobs succeeding? - Are users seeing slow or broken experiences? - Can we trace one business action end-to-end?

Google Cloud Observability is the right control plane for this system because Cloud Run int natively with Cloud Logging and Cloud Monitoring. (docs.cloud.google.com) (docs.cloud.google.com)

Observability pillars

Implement all four from the beginning: - structured logs - metrics - traces - alerts

Structured logging standard

Every log line should be JSON and include these fields `timestamp` lic `severity` - `service` `environment` `version` `requestId` `correlationId` `tenantId` `userId` - `providerType` `providerConnectionId` `transactionId` `documentId` `syncRunId` - `taskName` - `eventType` - `errorCode` - `latencyMs`

Logging rules: - never log plaintext secrets, tokens, passwords, or full provider credentials - r heavy payload fragments unless explicitly needed for support and legally permitted - log bus identifiers and storage references instead of large raw payloads - all async jobs must pre `correlationId` from parent request or sync run

Recommended log categories: - `api.request` - `api.response` - `auth.event` - `provider.webhook` `provider.poll` `provider.validation` `transaction.ingestion` - `document.issuance` - `task.execution` - `security.event`

Core service-level indicators

API SLIs

- request success rate
- p 95 and p 99 latency by endpoint group
- auth failure rate
- 5 xx error rate

Provider ingestion SLIs

- webhook verification success rate
- sync success rate per provider connection
- mean lag from upstream event receipt to canonical transaction visibility
- duplicate-suppression rate
- dead-letter rate

Document workflow SLIs

- issue-document success rate
- mean time to document issuance
- delivery success rate if delivery is enabled
- retry exhaustion rate

Data correctness indicators

- percentage of transactions with missing critical normalized fields
- number of duplicate conflicts per $1,000$ ingested records
- number of replayed payloads required per provider per week

Product indicators

- transaction list API latency
- transaction detail API latency
- mobile session refresh success rate
- login success rate

SLO recommendations for MVP

Initial practical targets: - API availability: 99.5% - p95 read latency for transaction list/deta 600 ms under normal load - successful provider sync runs: > 98% excluding upstrean provider outages - successful document issuance: > 97% excluding provider-side validation erro caused by bad input - auth login success for valid credentials: > 99%

These should be treated as operating targets, not contractual guarantees in early MVP.

Alerting policy

Cloud Monitoring alert policies should be configured for: - sustained API 5xx rate spike - p95 latency spike - failed login spike indicating abuse or incident - provider connection con failures above threshold - no webhook activity from a webhook-based provider beyond expect window - sync queue backlog above threshold - document issuance failure spike - dead-letter growth - secret access or auth anomalies where possible through audit logs

Cloud Run metrics and logs can be surfaced directly in Cloud Monitoring, which is the natu these alerts. (docs.cloud.google.com)

Tracing and correlation

Every external request and every internal task must be traceable end-to-end.

Correlation rules: - as `requestId` at API ingress if the client does not provide one - cre `correlationId` for long-running workflows such as sync runs and document issuance - propa both through Cloud Tasks payloads and Pub/Sub messages - audit events should store the `correlationId`

Minimum traceable chains: - login -> protected API call - webhook -> archival -> normaliza transaction write - manual sync trigger -> queue -> page processing -> sync completion - document -> queue -> provider call -> document write -> delivery status

Error budget policy

For each major SLO, track an error budget monthly.

Usage guidance: - if error budget burn is low, continue feature work - if error budget bur prioritize provider hardening and operational fixes - if one provider repeatedly dominates incide isolate it behind feature flags or reduced sync frequency until stabilized

Runbook set

Create runbooks before production for the top incidents.

Required runbooks: - provider credentials expired or invalid - webhook signature failures spike backlog growing - document issuance failures spike - login failures spike - Firestore hot path increase - mistaken duplicate suppression or missed transaction ingestion - manual replay of payload - rollback of bad release

Runbook template: - symptom - likely causes - dashboards/log queries to inspect - immedi containment steps - longer-term remediation - escalation owner

Dead-letter and replay operations

Cloud Tasks provides queue-level retry behavior and rate control, which should be used with retries and dead-letter-style operational review. (docs.cloud.google.com) (docs.cloud.google.com)

Operational rules: - retryable tasks use queue-defined exponential backoff - non-retryable mappin validation failures move to review state quickly - every dead-letter item includes raw payload normalized error code, and replay action metadata - replay actions require privileged admin r are fully audited

Reliability patterns

Use these patterns consistently: - idempotent task handlers - narrow retries with bounded ba queue separation by workload type - service-to-service authentication for internal invocations - immutable raw payload archive for replay - graceful degradation of non-critical features such analytics or push notifications

Cloud Run's private-by-default IAM model and documented service-to-service authentication make i suitable for locking down internal worker/task endpoints. docs.cloud.google.com) (docs.cloud.google.com)

Queue design

Recommended Cloud Tasks queue `provider-validation` `provider-sync` `document-issuance` - `notification-delivery` (later) - `replay-jobs` (admin-only)

Queue policy guidance: - configure retry behavior per queue according to workload criticality b queue-level retry settings apply to all Cloud Tasks-created tasks in that queue. (docs.cloud.google.com) (- keep lower concurrency on provider sync queues to respect vendor rate limits - allow faster document issuance if provider capacity permits - isolate replay work from live production sync traffic

Suggested starting retry policy: - provider-validation: max attempts `5`, moderate backoff - provi sync: max attempts `10` for upstream transient failures, slower backoff - document-issuance: ma attempts `7`, moderate backoff - replay-jobs: manual or very low retry, operator supervised

Deployment topology and infrastructure-as-code model

The deployment topology should stay simple for MVP but strongly separated by concern a environment.

GCP project layout

Recommended minimum projects: - `payment-app-staging` - `payment-app-production`

Optional later: - `payment-app-shared` for central CI artifacts, shared DNS, or org-level monitoring

Keep staging and production fully separated for: - Firestore database - Storage buckets - Cl services - Secret Manager secrets - Cloud Tasks queues - Pub/Sub topics/subscriptions - IAM policies

Runtime topology

Cloud Run services `api-service` public, authenticated by app bearer tokens for user calls anc provider webhooks where nee `worker-service` private, invoked by Cloud Tasks and optionally Pub/Sub push - `admin-web` public frontend hosting layer or static hosting plus API calls

Service identities: - dedicated service account per runtime - least-privilege IAM bindings - sep invoker permissions so only Cloud Tasks/PubSub or allowed services can call private worker endpoints

Cloud Run supports IAM-backed invocation and service identities, which should be used instea shared secrets between services where possible. (docs.cloud.google.com) (docs.cloud.google.com)

Networking and exposure

MVP networking posture: - `worker-service` private - expose `api-service` and the admin web frontend - use HTTPS everywhere through managed Google fronting - do not internal task or replay endpoints publicly

Secret and configuration injection

Use Secret Manager for: - JWT signing keys - provider credentials - email provider credentials OAuth secrets

Cloud Run documentation recommends using Secret Manager for sensitive service configuration. (docs.cloud.google.com)

Rules: - no secrets in source control - no secrets committed to Terraform variables files - r provider secrets with versioning support - reference secret versions explicitly where rollout con matters

Terraform module structure

Recommended Terraform repository structure:

```
/infra
  /modules
```

```
    /artifact_registry
    /service_accounts
    /cloud_run_service
    /firestore_indexes
    /storage_bucket
    /secret_manager_secret
    /cloud_tasks_queue
    /pubsub_topic
    /cloud_scheduler_job
    /monitoring_alerts
    /iam_bindings
  /envs
    /staging
      main.tf
      variables.tf
      outputs.tf
    /production
      main.tf
      variables.tf
      outputs.tf
```

Module responsibilities: `cloud_run_service`: container image, env vars, secret mounts, concurrency, CPU/memory, service account, invoker `firestore_indexes`: composite indexes and single-field exemptions `cloud_tasks_queue`: retry settings, rate limits, routing target - `monitoring_alerts`: SLO-relevant alert policies

Terraform security best practices from Google are especially relevant here: use remote state, storing secrets in state where possible, and run pre-apply checks. (docs.cloud.google.com)

CI/CD pipeline design

Recommended pipeline stages: 1. lint and unit tests 2. build container images 3. vulnerabil scan 4. push to Artifact Registry 5. deploy to staging 6. run smoke tests 7. manu approval for production 8. deploy to production 9. post-deploy smoke and health verification

Release policy: - backend and admin web can deploy independently - mobile releases follow cadence but should target backward-compatible API changes - use feature flags for unfinishe provider-specific logic

Runtime configuration guidance

Suggested Cloud Run starting set `api-service`: moderate concurrency, minimum instances 0 or 1 depending on cold-start tolerance, CPU/memory sized for webhook and API traffic `worker-service`: lower concurrency for provider-facing workloads, separate scaling policy from API

Rules: - do not share one service for every concern if concurrency and IAM profiles differ split services when scaling policy, security posture, or runtime dependency profile differs

Backup and recovery posture

MVP recovery posture should include: - Firestore export or backup strategy documented and sc where appropriate - Storage bucket lifecycle and retention policies for raw payloads and docu ability to redeploy infrastructure from Terraform - recovery of secrets through Secret Manager history - replay capability from archived payloads after partial data loss or mapping defect

Environment promotion policy

Promotion rules: - provider adapters must pass sandbox contract tests in staging before prod enablement - Firestore index changes must be applied before features that depend on them released - secret references must exist in target environment before deployment - rollout che must include webhook endpoint validation and queue health verification

Sequence-diagram pack for critical workflows

The following sequence diagrams are intended to remove ambiguity from the highest-risk syst behaviors. They should be treated as implementation reference flows for backend, mobile, admin and operations work.

1 . User registration and initial tenant bootstrap

```
@startuml
actor User
participant "Mobile App / Admin Web" as Client
participant "API Service" as API
database "Firestore" as Firestore
participant "Password Hasher" as Hasher
queue "Audit Events" as Audit

User -> Client : Enter email, password, tenant info
Client -> API : POST /v1/auth/register
API -> API : Normalize email
API -> Firestore : Lookup user by emailLower
alt Email already exists
  Firestore --> API : Found
  API --> Client : 409 auth.email_already_exists
else Email available
  Firestore --> API : Not found
  API -> Hasher : Hash password (Argon2id)
  Hasher --> API : passwordHash
  API -> Firestore : Create users/{userId}
  API -> Firestore : Create tenants/{tenantId}
  API -> Firestore : Create tenant_memberships/{tenantId_userId}
  API -> Firestore : Create auth_sessions/{sessionId}
  API -> Audit : auth.registered, tenant.created
  API --> Client : accessToken + refreshToken + tenant context
end
@enduml
```

2 . Login and refresh-token rotation

```
@startuml
actor User
participant "Mobile App / Admin Web" as Client
participant "API Service" as API
database "Firestore" as Firestore
participant "Password Hasher" as Hasher
queue "Audit Events" as Audit

User -> Client : Enter email and password
Client -> API : POST /v1/auth/login
API -> API : Normalize email
API -> Firestore : Find user by emailLower
alt User not found or locked
  API -> Audit : auth.login_failed
  API --> Client : 401/423
else User found
  Firestore --> API : user record
  API -> Hasher : Verify password
  alt Password invalid
    API -> Firestore : Increment failedLoginCount
    API -> Audit : auth.login_failed
    API --> Client : 401
  else Password valid
    API -> Firestore : Reset failedLoginCount
    API -> Firestore : Create auth_sessions/{sessionId}
    API -> Audit : auth.login_succeeded
    API --> Client : accessToken + refreshToken
  end
end

Client -> API : POST /v1/auth/refresh
API -> Firestore : Find session by sessionId / refreshTokenHash
alt Session revoked or expired
  API --> Client : 401 auth.invalid_session
else Session active
  API -> Firestore : Rotate refreshTokenHash
  API -> Audit : auth.refresh_succeeded
  API --> Client : new accessToken + new refreshToken
end
@enduml
```

3 . Password reset

```
@startuml
actor User
participant "Mobile App / Admin Web" as Client
```

```
participant "API Service" as API
database "Firestore" as Firestore
participant "Email Provider" as Email
queue "Audit Events" as Audit

User -> Client : Request password reset
Client -> API : POST /v1/auth/forgot-password
API -> API : Normalize email
API -> Firestore : Lookup user by emailLower
alt User exists
  API -> Firestore : Create password_reset_tokens/{tokenId}
  API -> Email : Send reset link with one-time token
  API -> Audit : auth.password_reset_requested
else User missing
  API -> Audit : auth.password_reset_requested
end
API --> Client : accepted=true

User -> Client : Submit new password with token
Client -> API : POST /v1/auth/reset-password
API -> Firestore : Lookup reset token by tokenId/hash
alt Invalid or expired token
  API --> Client : 400 auth.invalid_reset_token
else Valid token
  API -> API : Hash new password
  API -> Firestore : Update users/{userId}.passwordHash
  API -> Firestore : Revoke all auth_sessions for user
  API -> Firestore : Mark reset token used
  API -> Audit : auth.password_reset_completed
  API --> Client : passwordReset=true
end
@enduml
```

4 . Provider connection validation and activation

```
@startuml
actor Admin
participant "Admin Web" as AdminWeb
participant "API Service" as API
database "Firestore" as Firestore
participant "Secret Manager" as Secrets
queue "Cloud Tasks" as Tasks
participant "Worker Service" as Worker
participant "Provider Adapter" as Adapter
queue "Audit Events" as Audit

Admin -> AdminWeb : Enter provider credentials
AdminWeb -> API : POST /provider-connections
API -> Secrets : Store provider credentials
```

```
API -> Firestore : Create provider_connections/{connectionId}
status=validating
API -> Tasks : Enqueue validate-provider-connection
API --> AdminWeb : queued/validating

Tasks -> Worker : /internal/tasks/validate-provider-connection
Worker -> Firestore : Load provider connection
Worker -> Secrets : Fetch credentials
Worker -> Adapter : validate_credentials()
alt Validation failed
  Worker -> Firestore : Update connection status=error, health fields
  Worker -> Audit : provider.connection_failed
else Validation succeeded
  alt Provider supports webhook registration
    Worker -> Adapter : register_webhook()
    Adapter --> Worker : webhook registration result
  end
  Worker -> Firestore : Update connection status=active
  Worker -> Tasks : Enqueue initial sync
  Worker -> Audit : provider.connection_activated
end
@enduml
```

5 . Webhook intake to canonical transaction write

```
@startuml
actor Provider
participant "API Service" as API
participant "Provider Registry" as Registry
participant "Provider Adapter" as Adapter
folder "Cloud Storage" as GCS
queue "Cloud Tasks" as Tasks
participant "Worker Service" as Worker
database "Firestore" as Firestore
queue "Pub/Sub" as PubSub
queue "Audit Events" as Audit

Provider -> API : POST /v1/providers/{providerType}/webhook
API -> Registry : Resolve adapter
API -> Adapter : verify_webhook(headers, body)
alt Signature invalid
  API -> Audit : webhook.signature_failed
  API --> Provider : 401/403
else Verified or accepted
  API -> GCS : Archive raw payload
  API -> Audit : provider.webhook_received
  API -> Tasks : Enqueue process-sync-page / process-webhook-event
  API --> Provider : accepted=true
end
```

```
Tasks -> Worker : Process webhook payload
Worker -> GCS : Load archived payload if needed
Worker -> Adapter : parse_webhook()
Worker -> Adapter : normalize_event()
Worker -> Worker : derive dedup keys
Worker -> Firestore : Reserve transaction_dedup key
alt Duplicate found
  Worker -> Firestore : Load existing transaction
  Worker -> Firestore : Merge/no-op according to rules
  Worker -> Audit : transaction.duplicate_detected
else New transaction
  Worker -> Firestore : Create transactions/{transactionId}
  Worker -> Firestore : Create alias dedup keys
  Worker -> Audit : transaction.created
  Worker -> PubSub : Publish transaction.created
end
@enduml
```

6 . Scheduled polling sync with paginated cursor

```
@startuml
participant "Cloud Scheduler" as Scheduler
participant "API Service" as API
database "Firestore" as Firestore
queue "Cloud Tasks" as Tasks
participant "Worker Service" as Worker
participant "Secret Manager" as Secrets
participant "Provider Adapter" as Adapter
folder "Cloud Storage" as GCS
queue "Audit Events" as Audit

Scheduler -> API : POST /internal/sync/start
API -> Firestore : Query active provider connections
loop For each due connection
  API -> Firestore : Create sync_runs/{syncRunId}
  API -> Tasks : Enqueue process-sync-page with initial cursor
end

Tasks -> Worker : /internal/tasks/process-sync-page
Worker -> Firestore : Load sync run + connection
Worker -> Secrets : Load provider credentials
Worker -> Adapter : fetch_incremental(cursor)
loop For each provider record in page
  Worker -> GCS : Archive raw record
  Worker -> Adapter : normalize record
  Worker -> Firestore : Dedup reserve + create/merge transaction
end
alt More pages available
```

```
  Worker -> Tasks : Enqueue next page with updated tentative cursor
  Worker -> Firestore : Update sync run progress
else No more pages
  Worker -> Firestore : Advance durable connection cursor
  Worker -> Firestore : Mark sync run succeeded
  Worker -> Audit : provider.sync_succeeded
end
alt Upstream transient failure
  Worker -> Firestore : Update sync run partial/error state
  Worker --> Tasks : Retry by queue policy
end
@enduml
```

7 . Manual issue-document flow

```
@startuml
actor User
participant "Mobile App / Admin Web" as Client
participant "API Service" as API
database "Firestore" as Firestore
queue "Cloud Tasks" as Tasks
participant "Worker Service" as Worker
participant "Secret Manager" as Secrets
participant "Document Adapter" as DocAdapter
folder "Cloud Storage" as GCS
queue "Audit Events" as Audit

User -> Client : Issue receipt/document
Client -> API : POST /transactions/{id}/issue-document
API -> Firestore : Load transaction
API -> API : Validate allowed state
API -> API : Compute requestHash
API -> Firestore : Check existing documents by requestHash
alt Existing successful document
  API --> Client : return existing result
else No existing success
  API -> Tasks : Enqueue issue-document task
  API -> Audit : document.issue_requested
  API --> Client : queued=true
end

Tasks -> Worker : /internal/tasks/issue-document
Worker -> Firestore : Re-check document by requestHash
alt Already issued by prior attempt
  Worker --> Tasks : ack/no-op
else Needs issuance
  Worker -> Secrets : Load document provider credentials
  Worker -> DocAdapter : issue_document()
  alt Issuance success
```

```
    Worker -> GCS : Store document file if provided
    Worker -> Firestore : Create documents/{documentId}
    Worker -> Firestore : Update transactions/
{transactionId}.receiptState=issued
    Worker -> Audit : document.issued
  else Recoverable failure
    Worker -> Audit : document.issue_retryable_failure
    Worker --> Tasks : retry by queue policy
  else Terminal failure
    Worker -> Firestore : Create/update documents status=failed
    Worker -> Firestore : Update transaction receiptState=failed
    Worker -> Audit : document.issue_failed
  end
end
@enduml
```

8 . Duplicate-safe webhook + polling race condition

```
@startuml
participant "Webhook Worker" as WebhookWorker
participant "Polling Worker" as PollingWorker
database "Firestore" as Firestore
queue "Audit Events" as Audit

par Concurrent processing of same real-world payment
  WebhookWorker -> Firestore : Reserve transaction_dedup/{tenant_hash}
  alt Reservation succeeds
    WebhookWorker -> Firestore : Create transaction
    WebhookWorker -> Firestore : Write alias dedup keys
    WebhookWorker -> Audit : transaction.created
  else Reservation already exists
    WebhookWorker -> Firestore : Load existing transaction
    WebhookWorker -> Audit : transaction.duplicate_detected
  end
else
  PollingWorker -> Firestore : Reserve transaction_dedup/{tenant_hash}
  alt Reservation succeeds
    PollingWorker -> Firestore : Create transaction
    PollingWorker -> Firestore : Write alias dedup keys
    PollingWorker -> Audit : transaction.created
  else Reservation already exists
    PollingWorker -> Firestore : Load existing transaction
    PollingWorker -> Audit : transaction.duplicate_detected
  end
end
@enduml
```

9. Authenticated internal task invocation

```
@startuml
participant "Cloud Tasks" as Tasks
participant "Worker Service" as Worker
participant "Cloud Run IAM" as IAM
queue "Audit Events" as Audit

Tasks -> IAM : Mint OIDC token for service account
IAM --> Tasks : signed identity token
Tasks -> Worker : POST /internal/tasks/... + OIDC token
Worker -> Worker : Verify Cloud Run/IAM-authenticated invocation
alt Authorized caller
  Worker -> Audit : task.execution_started
  Worker -> Worker : Execute handler
  Worker -> Audit : task.execution_completed
  Worker --> Tasks : 200 OK
else Unauthorized
  Worker -> Audit : security.internal_invocation_rejected
  Worker --> Tasks : 401/403
end
@enduml
```

10. End-user transaction list fetch with cursor pagination

```
@startuml
actor User
participant "Mobile App" as Mobile
participant "API Service" as API
database "Firestore" as Firestore

User -> Mobile : Open transactions list
Mobile -> API : GET /transactions?limit=20&cursor=...
API -> API : Validate access token and tenant membership
API -> API : Validate supported filter combination
API -> Firestore : Query approved composite index
Firestore --> API : Page of transactions + last doc
API -> API : Build nextCursor
API --> Mobile : data[] + nextCursor + hasMore

alt User scrolls further
  Mobile -> API : GET /transactions?cursor=nextCursor
  API -> Firestore : Continue query with startAfter cursor
  Firestore --> API : Next page
  API --> Mobile : data[] + nextCursor
end
@enduml
```

Threat model and security review pack

This section applies a practical threat model to the architecture using system decomposition, boundaries, likely threat scenarios, and concrete mitigations. The goal is to reduce the real multi-tenant payment-orchestration application rather than produce a generic security checklist.

Security objectives

Primary security objectives: - preserve tenant isolation at all times - prevent unauthorized acc payment, customer, and document data - prevent forgery or replay of provider events and d issuance actions - protect credentials, session tokens, and secrets - preserve auditability and repudiation for business-critical operations - keep operational recovery possible without weakening security controls

Data classification

Classify system data into four levels.

Level 1 : Public or low sensitivity

Examples: - marketing content - non-sensitive application metadata

Controls: - normal integrity controls - no special storage restrictions

Level 2 : Internal operational data

Examples: - non-sensitive provider health status - sync run statistics - feature flags

Controls: - authenticated access only - logged access for admin/support workflows

Level 3 : Confidential business and customer data

Examples: - transaction records - customer names, phone numbers, emails - document issuance audit events containing business context

Controls: - strict tenant isolation - least-privilege access - redaction in logs - minimal exposur payloads

Level 4 : Highly sensitive secrets and authentication material

Examples: - password hashes - refresh token hashes - JWT signing keys - provider API crede provider credentials - future OAuth secrets

Controls: - Secret Manager for secret material - never return in APIs - never log - highly explicit rotation procedures

Trust boundaries

Key trust boundaries in the system: - end-user client to public API - provider webhook call API - API service to worker service - runtime services to Firestore - runtime services to Se

runtime services to Cloud Storage - admin users to support/operations functions - staging envi to production environment

Every boundary must have an explicit authentication and authorization strategy.

System actors

Trusted or semi-trusted actors: - mobile users - admin/support users - backend API service - worker service - Cloud Tasks - Pub/Sub - Secret Manager - provider systems - document systems

Potential adversaries: - unauthenticated internet attacker - authenticated but malicious tenant use compromised provider endpoint or spoofed webhook sender - compromised mobile device - ins with excessive admin privileges - contractor with staging access attempting production pivot - attempting credential stuffing or token replay

Primary attack surfaces

External attack surfaces: - login endpoints - password reset endpoints - provider webhook end public admin-web and mobile API traffic

Internal attack surfaces: - internal task endpoints - service account permissions - secret access replay/admin operational endpoints - CI/CD deployment pipeline

Data-layer attack surfaces: - Firestore tenant-scoped queries - GCS raw payload/document access data exposure

Threat scenarios and mitigations

1. **Credential stuffing against email/password login**

Threat: - attacker attempts reused passwords or automated login attempts

Impact: - unauthorized tenant access - data leakage - fraudulent document issuance

Mitigations: - strong password hashing with Argon2id - rate limiting by IP and normalized failed login counters and temporary lockout - generic login failure messages to avoid acco enumeration - audit events for repeated failed logins - optional future MFA or verified second high-risk actions

2. **Password reset abuse and account takeover**

Threat: - attacker tries to enumerate users or replay reset tokens

Mitigations: - always return generic accepted response for reset requests - store only reset-t hashes, never plaintext tokens - short-lived one-time reset tokens - revoke existing sessions successful password reset - log all reset lifecycle events - require recent authentication for e sensitive account changes later

3 **. Refresh-token theft or replay**

Threat: - stolen refresh token from compromised device or intercepted storage

Mitigations: - store only refresh-token hashes server-side - rotate refresh token on each succ refresh - session-level revocation - logout-all and password-change revocation support - short-live access tokens to reduce window of abuse - store device/session metadata for detection and user review

4 **. Tenant isolation failure**

Threat: - a user accesses another tenant's records through insecure direct object references, bugs, or missing authorization checks

Mitigations: - backend authorization on every protected endpoint - tenant scope taken from va membership, not only client-supplied headers - repository/service methods require tenant context explicitly - test suite includes negative cross-tenant access cases for every resource type - support break-glass access, if introduced, must be tightly scoped and audited

5 **. Webhook spoofing or forgery**

Threat: - attacker posts fake provider events to create false transactions or trigger workflows

Mitigations: - provider-specific signature verification where supported - secret callback path or sh secret validation if provider lacks signing - rate limiting and anomaly logging for webhook er archival of original request bytes and verification result - no direct financial or irreversible act webhook without canonical validation and dedup checks

6 **. Webhook replay attack**

Threat: - valid webhook payload resent multiple times

Mitigations: - dedup based on provider event ID / transaction ID / fingerprint - idempotent handlers - audit repeated deliveries - optional replay window checks if provider includes sig timestamps

7 **. Polling cursor corruption or sync poisoning**

Threat: - bug or attacker manipulates cursor state causing missed records or repeated ingestion

Mitigations: - cursor treated as opaque and only advanced after successful page commit - syn separate from durable committed cursor - replay support from archived payloads - operator-visib statistics and anomaly alerts

8 **. Unauthorized internal endpoint invocation**

Threat: - attacker or misconfigured service calls worker endpoints directly

Mitigations: - keep worker service private - require IAM-authenticated service-to-service invocation validate caller identity on internal routes - separate service accounts by role - log and alert internal invocations

9. Secret leakage

Threat: - provider credentials, JWT keys, or email credentials leak through code, logs, Terraform debug tooling

Mitigations: - store secrets in Secret Manager - pin secret versions where rollout control matt secrets in source control and CI variables where possible - never log sensitive headers or c tightly scope service-account access to only required secrets - maintain rotation procedure ar emergency revocation procedure

10. Excessive privilege in IAM or admin tooling

Threat: - service account or admin user can access more data or operations than required

Mitigations: - least-privilege IAM per service - separate service identities for API, worker, CI/CD admin ops - role-based authorization in the application - privileged replay/export actions restrict admin roles and fully audited - periodic permission review checklist

11. Raw payload or document exposure from Cloud Storage

Threat: - archived payloads or generated documents become publicly accessible or over-shared internally

Mitigations: - private buckets only - signed URLs generated only on demand and short-lived paths not treated as authorization - bucket IAM limited to required services - lifecycle/retention documented

12. Log-based data leakage

Threat: - PII, tokens, or secrets leak into logs and become broadly visible

Mitigations: - structured logging with redaction rules - exclude passwords, tokens, full credentials raw payload bodies from normal logs - log references to payload archive objects instead of - review log samples in staging before production

13. Provider-side validation abuse in document issuance

Threat: - attacker or buggy client repeatedly attempts invalid document issuance or manipula recipient fields

Mitigations: - server-side validation of issuance input - idempotency keys plus request hashing terminal validation errors do not retry indefinitely - delivery recipient validated against allowed and business rules - operator actions audited with actor identity

14. Mobile device compromise or token extraction

Threat: - attacker gains access to a user’s device storage or active session

Mitigations: - prefer secure platform storage for refresh tokens - short-lived access tokens - s session revocation support - avoid storing sensitive business datasets unencrypted in persistent storage - remote logout via session revocation

15. CI/CD or infrastructure misconfiguration

Threat: - pipeline deploys secrets incorrectly, exposes services publicly, or applies unsafe IAM changes

Mitigations: - Terraform changes reviewed in pull requests - environment separation between sta and production - production deploy requires approval gate - infrastructure smoke checks after policy checklist for public exposure, service identities, secrets, and queues

Abuse-case review

Important abuse cases to test explicitly: - tenant A user requests tenant B transaction by ID same webhook payload many times - call internal task endpoint without Cloud Run auth - document request repeatedly with same and different idempotency keys - reuse a revoked re token - use expired reset token - create provider connection with invalid credentials repeatedly large exports or list queries to force resource exhaustion

Security controls by layer

Client layer

- store refresh token in secure platform storage where available
- avoid storing raw payloads or provider credentials on device
- enforce TLS only
- clear sensitive session state on logout

API layer

- strict input validation via typed schemas
- authentication and tenant authorization on every protected endpoint
- rate limiting for auth and webhook endpoints
- idempotency support for risky mutations
- correlation IDs and security audit events

Worker layer

- private invocation only
- idempotent handlers
- replay-safe dedup logic
- no trust in task payload beyond identifiers and metadata

Data layer

- Firestore queries always tenant-scoped
- secret material kept out of Firestore
- storage buckets private by default
- short-lived signed URLs for document access

Infrastructure layer

- dedicated service accounts
- least-privilege IAM
- Secret Manager for secrets
- environment separation and reviewed IaC changes

Security validation and test plan

Required test categories: - auth and session tests - cross-tenant authorization tests - webhook validation tests - webhook replay tests - idempotency and duplicate suppression tests - inte endpoint auth tests - secret redaction/logging tests - password reset lifecycle tests - signed U tests - privilege/role matrix tests

Recommended additional validation: - dependency vulnerability scanning in CI - lightweight applica security review before launch - periodic manual review of high-risk flows after each new p integration

Security logging requirements

Security-significant events that must be logged: - login success/failure - lockout triggered - pas reset requested/completed - session revoked - provider connection created/updated/deleted - webho verification failed - internal invocation rejected - privileged replay/export action taken - secret anomalies if surfaced by platform logs

Incident response preparation

Minimum incident classes: - suspected account takeover - provider credential leak or compromi webhook forgery or replay campaign - cross-tenant data exposure bug - document exposure or URL misuse - deployment misconfiguration exposing private service

For each class define: - detection source - immediate containment action - owner - communic - recovery steps - postmortem requirement

Security decisions intentionally deferred from MVP

Deferred by design: - mandatory MFA for all users - dedicated WAF beyond managed cloud unless abuse indicates need - customer-managed encryption keys for all datasets - hardware-ba device binding - advanced anomaly detection or behavioral risk scoring

These can be added later without re-architecting the system, provided the core controls abov implemented correctly.

Security acceptance criteria

The architecture passes the MVP security review when: - all public endpoints have explicit au behavior - all internal endpoints require service-to-service authentication - tenant isolation is enf and tested across all major resource types - password, reset, and session materials are store secrets are managed outside source code and outside Firestore - webhook verification and r

protection exist for each provider - privileged operational actions are role-restricted and audited and responses do not leak secrets or unnecessary PII

Key architectural risks and mitigations

Risk: Firestore query limits for finance-style filtering. Mitigation: support a tightly defined filter denormalize search fields, and add export/report pipelines rather than ad hoc querying.

Risk: Duplicate transactions from mixed webhook and polling ingestion. Mitigation: dedicated ded reservation collection plus immutable audit events.

Risk: Provider inconsistency. Mitigation: adapter isolation, raw payload archival, provider-specific heal tracking, and replayable sync runs.

Risk: Receipt provider failure causing operational backlog. Mitigation: idempotent issuance tasks, exponential retries, dead-letter review queue, and operator retry controls.

Risk: Internal endpoint exposure or confused-deputy service calls. Mitigation: keep worker endpoi private and enforce IAM-backed service-to-service invocation.

Risk: Terraform leaks or secret sprawl. Mitigation: use Secret Manager, avoid secrets in state possible, separate environment access, and keep infra changes in reviewed pipelines.

Implementation

The implementation should prioritize a thin but production-credible MVP. Build the system in slices so contractors can deliver usable functionality early.

Delivery principles

- Build the backend API and admin web first for operational control.
- Build the mobile app in parallel once authentication, transaction listing, and transaction detail APIs are stable.
- Prefer direct integrations with managed GCP services rather than self-hosted infrastructure.
- Keep provider integrations behind explicit adapter boundaries from the first commit.
- Ship one payment source and one document provider first, then add the second payment source after the pipeline is stable.

Recommended repo strategy

Use a monorepo with three deployable apps: - `backend` - `mobile` - `admin-web`

Recommended branch/environment flov `main` -> productio `develop` -> staging - short-lived feature branches -> preview/test

Environment layout

Local development

- FastAPI runs locally.
- Firestore emulator is used where practical.
- Cloud Storage can be mocked locally or replaced with a development bucket.
- Task execution can run in synchronous/dev mode behind a feature flag.
- Mobile app points to local or staging backend.

Staging

- Separate GCP project.
- Separate Firestore database and Storage bucket.
- Separate Secret Manager secrets.
- Real Cloud Run, Cloud Tasks, Pub/Sub, Cloud Scheduler.
- Sandbox credentials for payment/document providers.

Production

- Dedicated GCP project.
- Locked-down secrets and IAM.
- Real provider credentials.
- Monitoring, alerting, backup/export procedures, and audit retention enabled.

GCP services to provision first

Core runtime

- Cloud Run service for API.
- Cloud Run service for worker.
- Firestore database.
- Cloud Storage bucket for raw payloads and documents.
- Secret Manager for provider credentials and application secrets.

Asynchronous processing

- Cloud Tasks queue for sync tasks.
- Cloud Tasks queue for document issuance.
- Pub/Sub topic for domain events.
- Cloud Scheduler jobs for recurring sync starts.

Operations and security

- Cloud Logging.
- Cloud Monitoring alert policies.
- IAM service accounts per service.
- Artifact Registry for container images.
- CI/CD pipeline to build and deploy containers.

Backend implementation order

Phase 1 : project foundation

Build: - FastAPI application skeleton - settings/config management - structured logging - heal endpoints - Firestore access layer - GCS client wrapper - Secret Manager wrapper - token s password hashing utilities

Recommended Python libraries: - FastAPI - Pydantic - httpx - passlib or pwdlib for password PyJWT or python-jose for token handling - google-cloud-firestore - google-cloud-storage - google-cl tasks - google-cloud-pubsub

Deliverables: - running API container - deployable Cloud Run manifest or Terraform - base ap and test harness

Phase 2 : authentication and tenant bootstrap

Build: - user registration with email/password - login endpoint - refresh token flow - logout/re tenant creation and first owner assignment - tenant membership lookup - role middleware/deper checks

Suggested auth model: - access token short-lived - refresh token longer-lived and rotatable - token metadata stored in Firestore - passwords hashed with Argon 2 or bcrypt

Collections to add/extend: - `users` - `tenant_memberships` - `auth_sessions`

Suggested `auth_sessions/{sessionId}` fields: `sessionId` - `userId` - `tenantId` - `refreshTokenHash` - `deviceInfo` - `ipAddressLastSeen` - `createdAt` - `expiresAt` - `revokedAt`

Deliverables: - mobile and web login working - protected endpoints working - tenant-aware a control enforced

Phase 3 : transaction ingestion pipeline

Build: - provider connection create/update APIs - adapter registry - webhook endpoint base - task start endpoint - raw payload archival - canonical normalization pipeline - dedup reservatio transaction write path - sync run tracking - audit event writing

Implementation note: - use Firestore transaction only for dedup reservation + canonical writ coordination where truly needed - avoid broad multi-document transaction usage

Deliverables: - first payment provider ingesting into canonical transactions - searchable transaction and detail endpoints - sync run visibility in admin web

Phase 4 : document issuance

Build: - document provider adapter - issue-document API - issue-document Cloud Task handle idempotent request hashing - document storage/archive flow - transaction receipt state updates failure and retry handling

Deliverables: - first compliant document provider working - operator can issue document from or admin web - failure states visible and retryable

Phase 5 : mobile app MVP

Build Flutter features in this order: - auth - session persistence - transaction list - transacti transaction details - issue document action - provider health/status view - settings/profile

Technical recommendations: - use Dio or equivalent HTTP client - local storage for session ar cache state - repository layer over API services - clear separation of presentation, domain, a concerns

Deliverables: - internal beta mobile app usable for real operator workflows

Phase 6 : admin web MVP

Build: - login - tenant settings - provider connection management - transaction lookup and support view - failed sync/document review - audit trail view

Deliverables: - operations/support staff can manage onboarding and failures without engineering involvement

Phase 7 : reliability and operations hardening

Build: - alerting for sync failure spikes and document failure spikes - dead-letter review pat unrecoverable jobs - replay tools for specific sync runs or payloads - export endpoints/files fo operations - rate limiting and abuse protection for auth and webhook endpoints - backup/ex strategy for Firestore and document assets

Deliverables: - production readiness baseline - runbooks for common incidents

API implementation details

Auth endpoints

- `POST /v1/auth/register`
- `POST /v1/auth/login`
- `POST /v1/auth/refresh`
- `POST /v1/auth/logout`
- `GET /v1/me`

Provider management endpoints

- `GET /v1/tenants/{tenantId}/provider-connections`

- `POST /v1/tenants/{tenantId}/provider-connections`
- `PATCH /v1/tenants/{tenantId}/provider-connections/{connectionId}`
- `POST /v1/tenants/{tenantId}/provider-connections/{connectionId}/validate`
- `POST /v1/tenants/{tenantId}/provider-connections/{connectionId}/sync`

Operations/internal endpoints

- `POST /internal/tasks/process-sync-page`
- `POST /internal/tasks/issue-document`
- `POST /internal/tasks/retry-document`
- `POST /internal/events/transaction-created`

Infrastructure as code

Use infrastructure as code from the beginning.

Recommended scope: - Cloud Run services - service accounts and IAM bindings - Firestore i Cloud Storage buckets - Cloud Tasks queues - Pub/Sub topics/subscriptions - Cloud Scheduler Secret Manager secrets - monitoring alerts

Recommendation: - use Terraform for GCP resources - keep application configuration in environ variables and secret references

Testing strategy

Backend

- unit tests for adapters, normalization, dedup logic, and auth services
- API tests for role enforcement and core endpoints
- emulator/integration tests for Firestore-backed flows
- contract tests for provider adapters using sandbox payloads

Mobile

- widget tests for core screens
- repository tests against mocked API
- one end-to-end happy path for login -> list transactions -> issue document

Admin web

- component tests for key forms and failure views
- API integration tests for onboarding and provider management

Data migration strategy

Plan now for future auth expansion.

MVP auth model: - email is the unique login identifier - user profile stores optional phone does not use it for sign-in yet

Future migration path: - add Google OAuth as an additional identity binding on the same u add phone verification fields to users - later support OTP login by linking verified phone n existing users - keep internal `userId` stable so business data does not need to migrate

Suggested additional user fields: - `passwordHash` - `passwordHashVersion` - `passwordChangedAt` - `googleAccountId` - `phoneNormalized` - `phoneVerified` - `phoneVerifiedAt` - `emailVerified` - `emailVerifiedAt` - `authMethods` `string[]`

Contractor handoff checklist

Contractors should receive: - this architecture spec - normalized transaction schema examples - provider payloads from the target integrations - UI wireframes for mobile and admin flows - matrix - environment access model - definition of done for each module - staging deploymen alert/runbook expectations

Milestones

Milestone 1 : Foundation

- GCP projects created for staging and production
- Cloud Run deployment working
- Firestore, Storage, Secret Manager, and logging connected
- FastAPI skeleton deployed

Milestone 2 : Auth and Tenant Setup

- Email/password registration and login complete
- Session and refresh token flow complete
- Tenant bootstrap and role enforcement complete
- Mobile and admin can access protected APIs

Milestone 3 : First Payment Integration

- First provider adapter complete
- Webhook and/or polling sync complete
- Transactions normalized and deduplicated
- Transaction list/detail available in admin and API

Milestone 4 : First Document Provider

- Document issuance flow complete
- Idempotent retry behavior complete
- Issued document state visible in transaction detail

Milestone 5 : Mobile MVP Beta

- Flutter app supports login, list, detail, and issue-document flow
- Basic provider status and settings screens available
- Internal beta with real business users starts

Milestone 6 : Second Payment Integration

- Second provider adapter complete
- Shared adapter framework validated
- Operational support workflows improved

Milestone 7 : Production Readiness

- Monitoring and alerting active
- Dead-letter/replay tooling available
- Exports and audit visibility complete
- Security review and launch checklist complete

Gathering Results

Success should be measured against both business usefulness and operational correctness.

Product success indicators

- Time required for an operator to find a transaction and issue a document
- Percentage of transactions that enter the system without manual intervention
- Percentage of document issuance requests completed successfully
- Number of support cases caused by missing or duplicate transactions

Operational indicators

- Sync success rate per provider
- Duplicate detection rate and false-duplicate rate
- Average time from payment event arrival to visible transaction in app
- Average time from document request to issued document
- Retry volume and dead-letter volume
- API latency for list/detail endpoints

Quality validation methods

- Compare imported transactions against provider dashboards during pilot rollout
- Reconcile a sample of issued documents with provider and invoicing records
- Review audit trails for all failed and retried workflows
- Run pilot tenants through webhook outage and retry scenarios

Post-production review cadence

- Daily review during pilot phase
- Weekly review after initial launch
- Monthly review of provider reliability, search performance, and auth/conversion metrics

Exit criteria for moving beyond MVP

- At least two stable payment integrations in production
- Document issuance success rate consistently high
- Low rate of duplicate/missed transaction incidents

- Admin team can resolve most operational failures without engineering intervention
- Auth model ready to expand from email/password to additional methods without changing core user IDs

Need Professional Help in Developing Your Architecture?

Please contact me at sammuti.com :)